香 港 科 技 大 學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC 3310
Digital Fundamentals and System Design

Wenchao Miao

Outline Lecture 1 Introductory Concepts

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Main Content

1. Basic Concepts of Digital Fundamentals and System Design

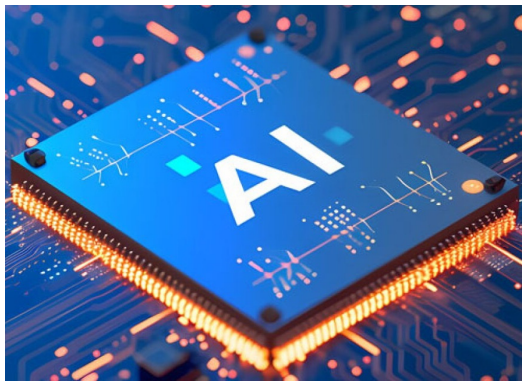
2. Learning Outcomes

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

1. Digital Fundamentals and System Design

Why learning Digital Fundamentals and System Design is important ?

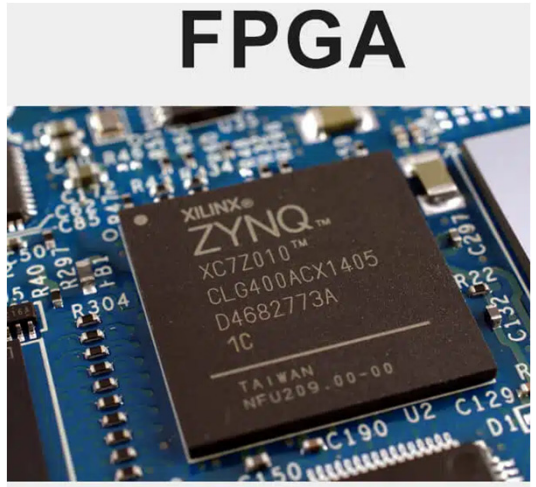
Digital systems form the backbone of modern technology—from smartphones and computers to advanced robotics, AI, and IoT devices. This course provides the essential foundation for designing and understanding these systems, making it crucial for future engineers and computer scientists

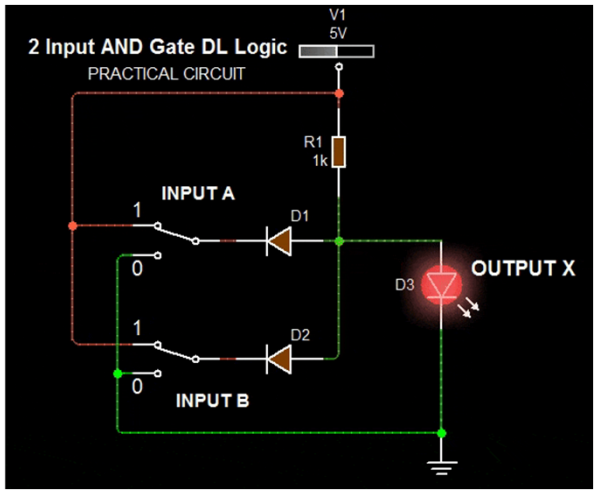


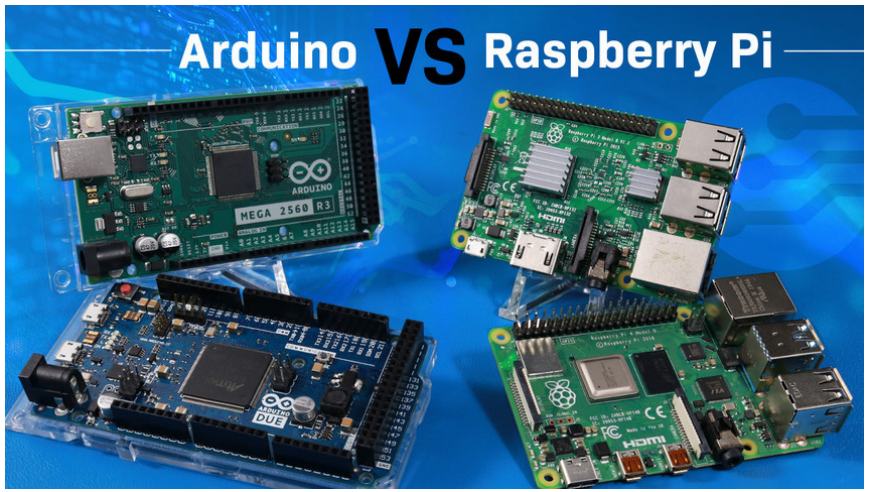
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(1) Core Knowledge for Hardware & Embedded Systems

- Digital logic is the basis of microprocessors, FPGAs, and embedded systems
- Understanding how logic gates, registers, and state machines work is essential for designing efficient hardware
- Many industries (automotive, aerospace, consumer electronics) rely on digital systems



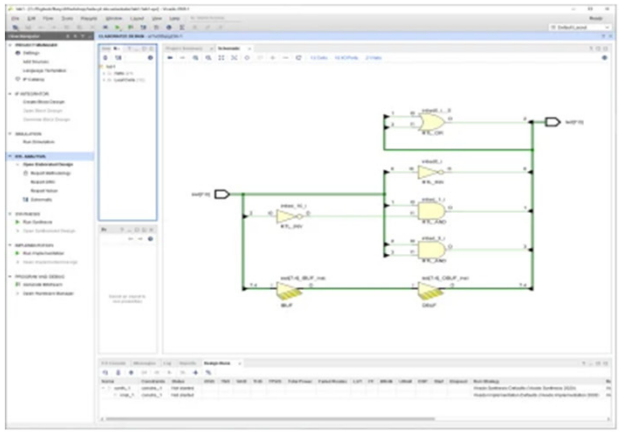


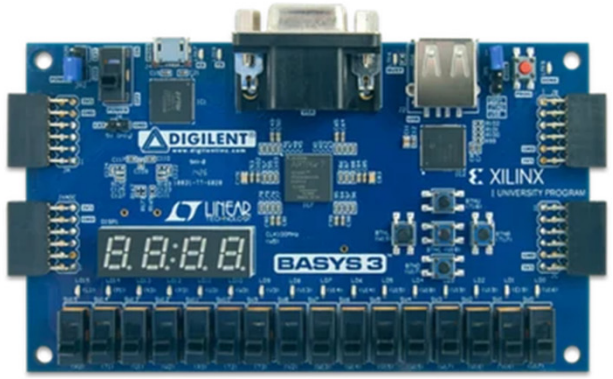


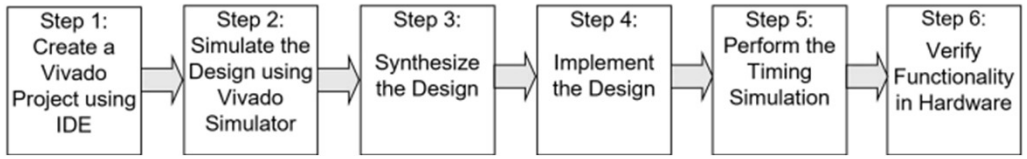
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

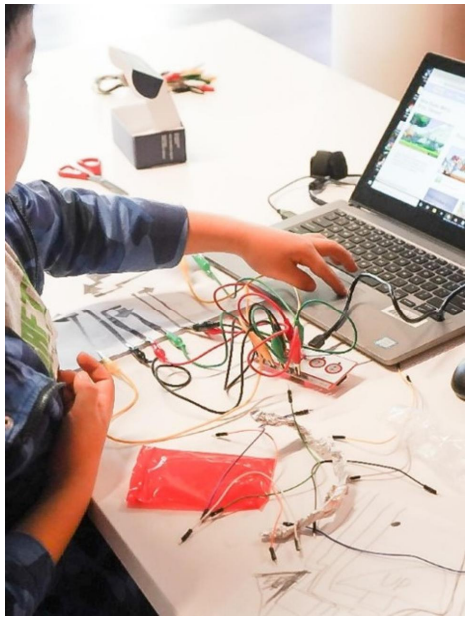
(2) Bridging Software and Hardware

- Modern computing requires hardware-software co-design (e.g., optimizing algorithms for FPGAs)
- Learning Hardware Description Languages (HDLs) like Verilog/VHDL helps you write code that directly controls hardware







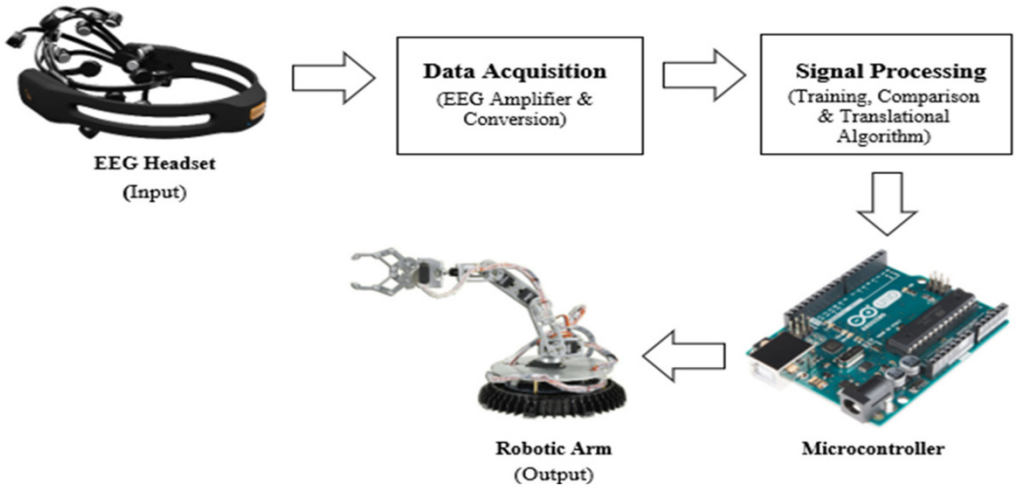


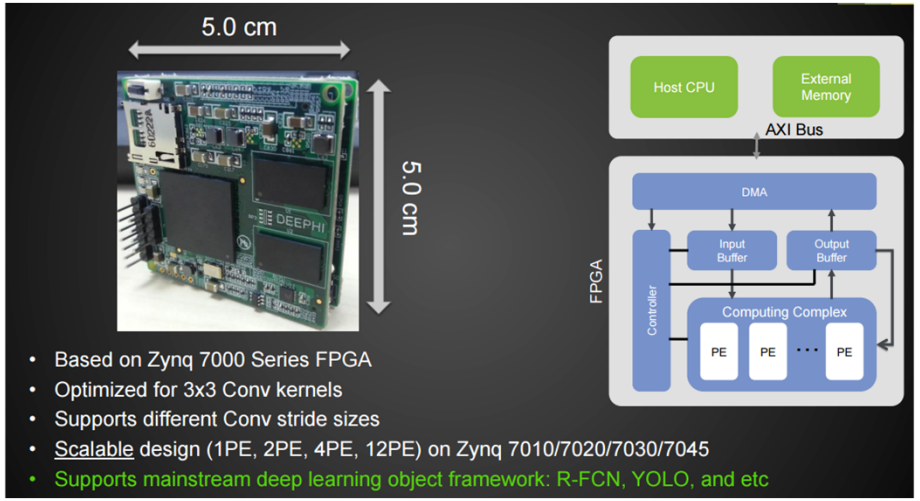
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(3) Foundation for Advanced Computing

Concepts from this course are used in:

- **Computer Architecture(CPU design, memory systems)**
- **Robotics & Control Systems(sensor interfacing, motor control)**
- **AI & Machine Learning(accelerators, custom ASICs/FPGAs for deep learning)**





THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(4) Hands-On Problem-Solving Skills

- We’ ll design and simulate real digital circuits
- Work with FPGA boards (used in industries for prototyping)
- Debug and optimize circuits—skills highly valued in R&D and electronics design

(5) Career Opportunities

- VLSI (Chip Design) Engineer– Design processors at Intel, AMD, NVIDIA
- Embedded Systems Engineer– Work on IoT, automotive systems
- FPGA Developer– Used in telecom, defense, and AI acceleration
- Hardware Security Engineer– Protect systems from cyber threats

Whether you’re into AI, robotics, chip design, or IoT, mastering digital logic is a must
This course isn’t just about theory—it’s about building real systems that power the technology of tomorrow

2. Learning Outcomes

1. Explain the basic differences between digital and analog quantities
2. Show how voltage levels are used to represent digital quantities
3. Describe various parameters of a pulse waveform such as rise time, fall time, pulse width, frequency, period, and duty cycle
4. Explain the basic logic functions of NOT, AND, and OR
5. Describe several types of logic operations and explain their application in an example system
6. Describe the basic laws and rules of Boolean algebra
7. Apply DeMorgan's theorems to Boolean expressions

2. Digital vs. Analog

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Main Content

Digital vs. Analog: Principles and Advantages

1. Core Definitions
2. Real-World Comparisons

2. Digital vs. Analog

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(1) Analog (Continuous) vs. Digital (Discrete) Signals

- Analog signal – continuous values (infinite possibilities)
- Digital signal – discrete values (finite set of values)

Discrete values


Continuous values

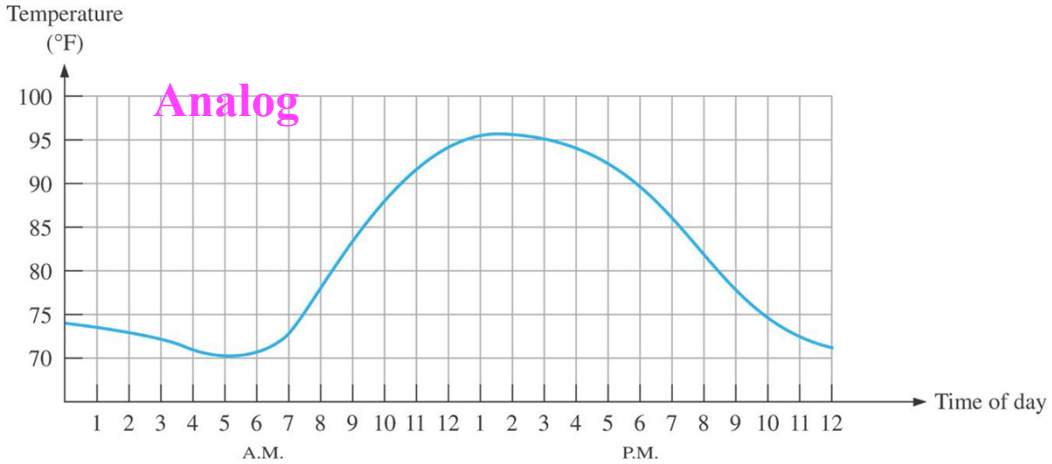

Secure Digital Card (SD Card)


Cassette Tape

- Digital data can be processed and transmitted more efficiently and reliably than analog data
- Digital data storage has a great advantage. Music can be stored more compactly and reproduced with greater accuracy and clarity in digital form than possible in analog form
- Noise (unwanted voltage fluctuations) does not affect digital data nearly as much as it does analog signals

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

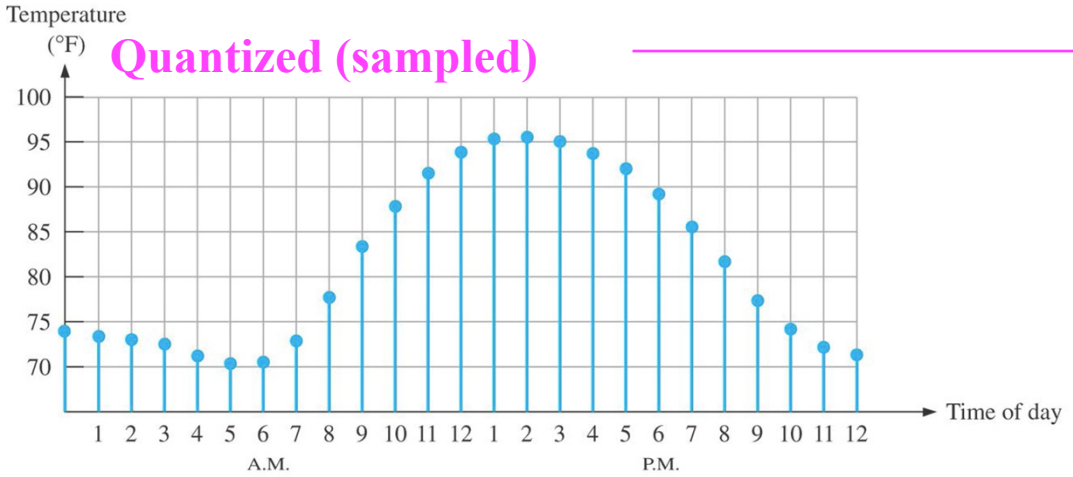
(2) Quantization & Digitization of Analog Data



Discrete values

Continuous values

Sampled-value representation (quantization) of the analog quantity. Each value represented by a dot can be digitized by representing it as a digital code that consists of a series of 1s and 0 s.

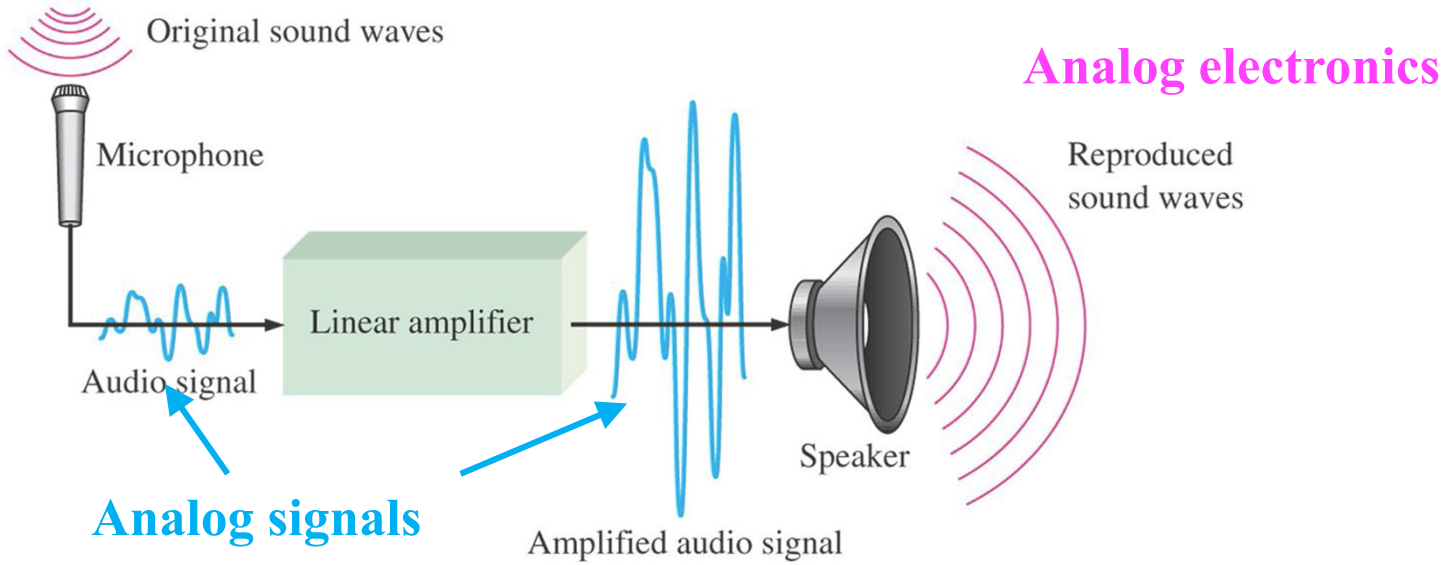


THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2. Real-World Comparisons

(1) Analog vs. Digital Audio Systems (Microphone vs. CD Player)

An audio system for addressing audience

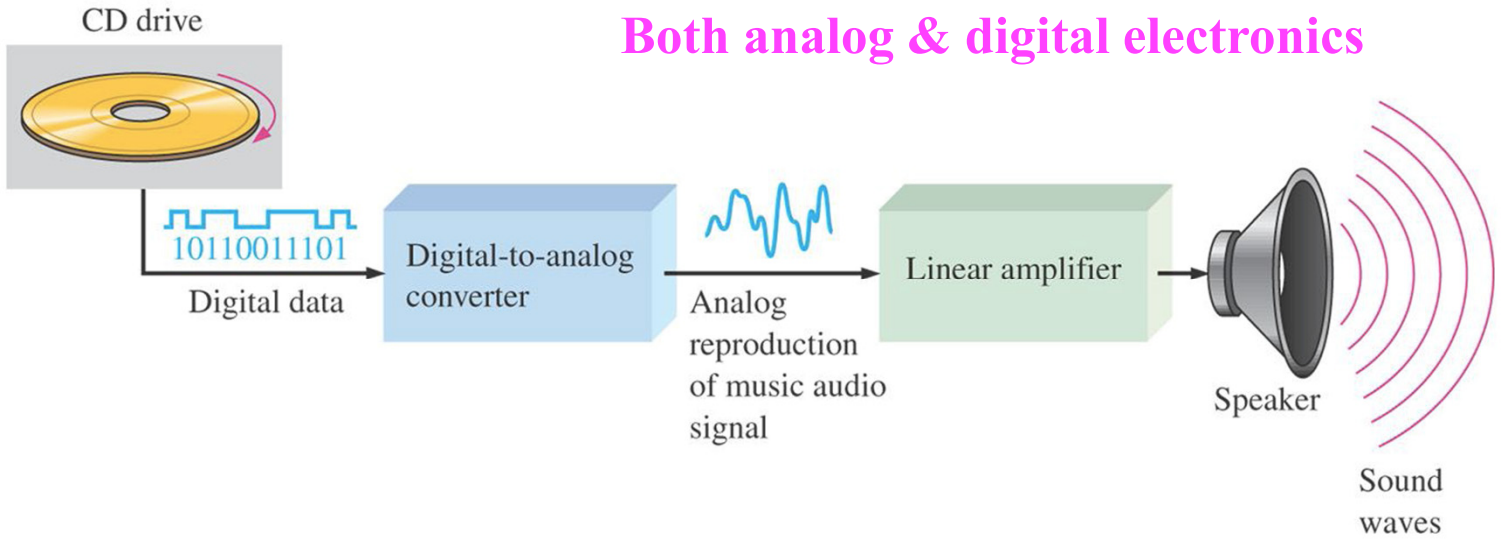


THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2. Real-World Comparisons

(1) Analog vs. Digital Audio Systems (Microphone vs. CD Player)

A basic diagram for a CD player system

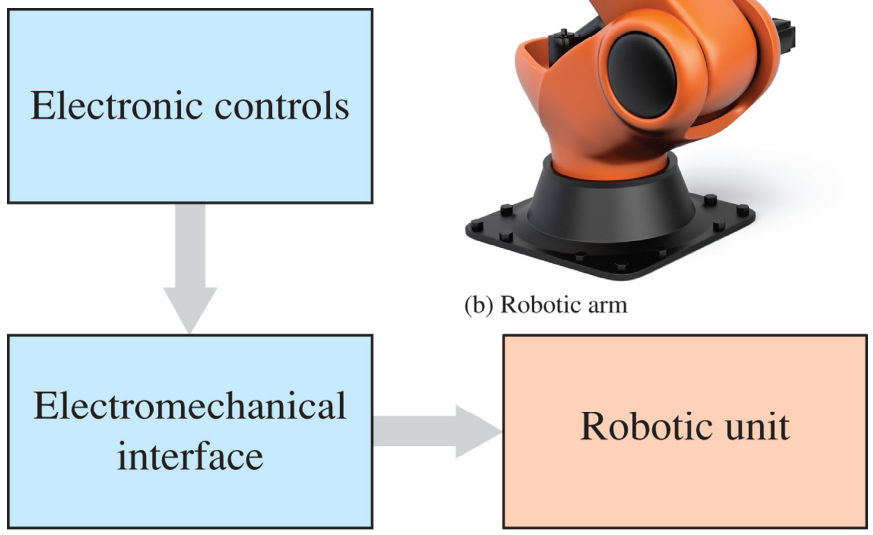


THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2. Real-World Comparisons

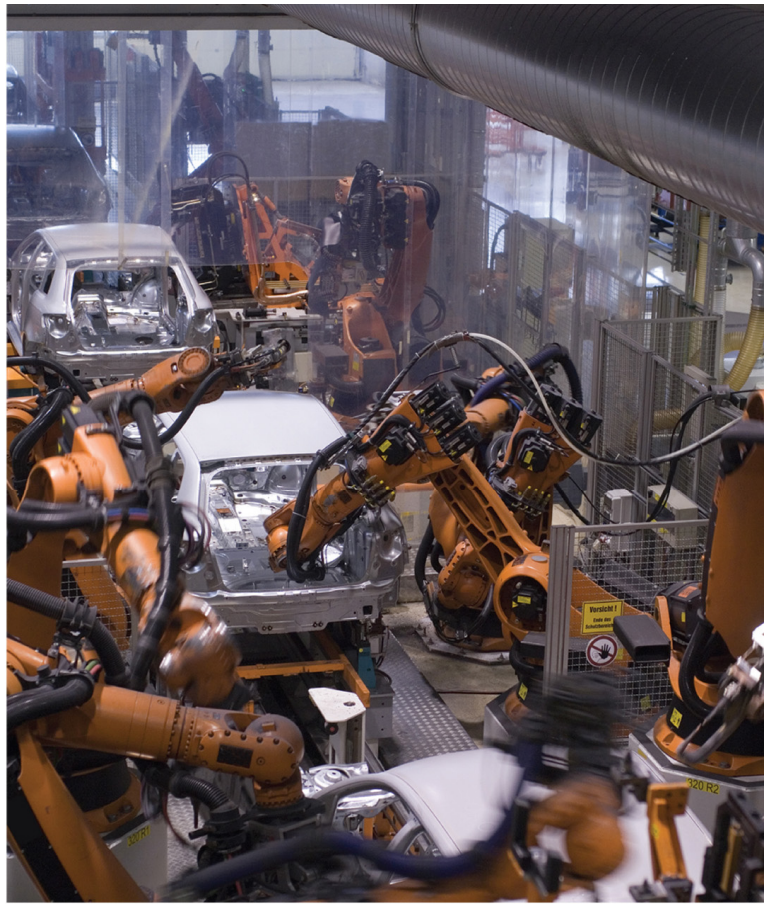
(2) Hybrid Systems in Mechatronics (Robotic Arm Example)

Both analog and digital systems are used in Mechatronics (robotic systems)



(a) Mechatronic system block diagram

(b) Robotic arm



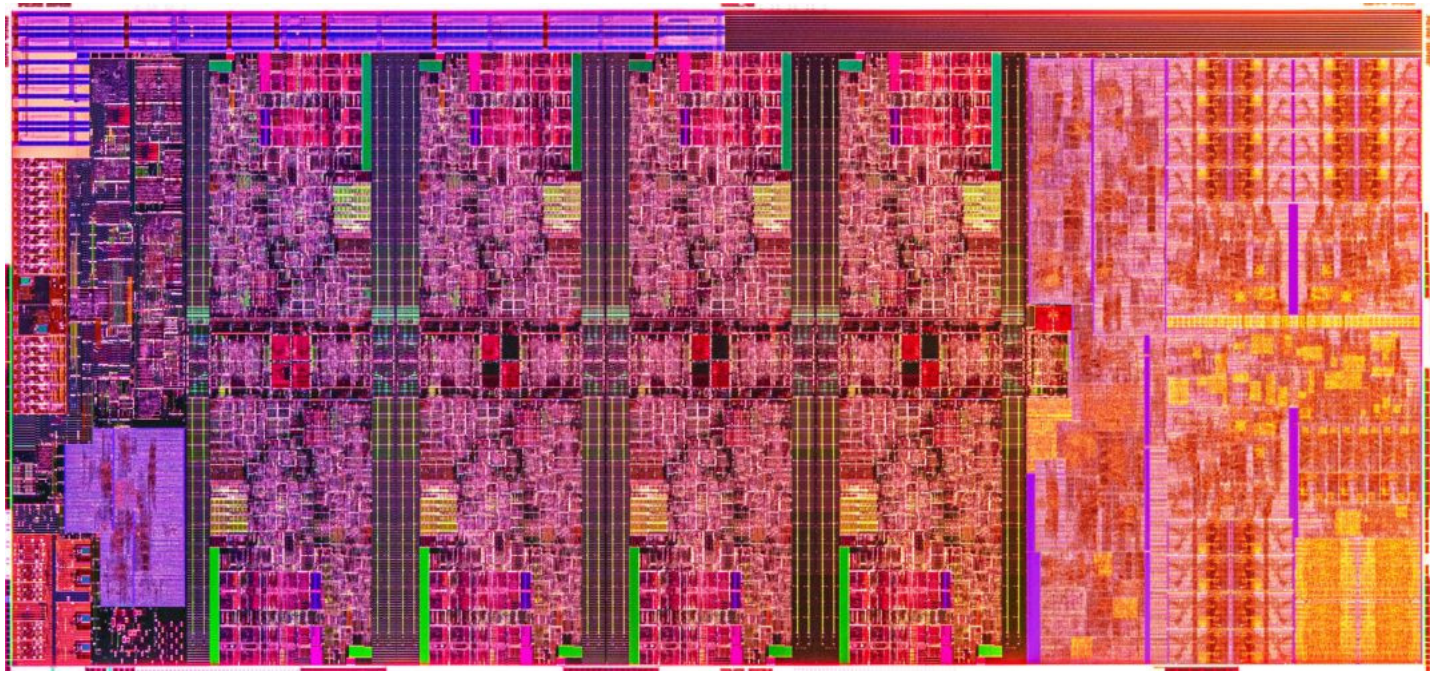
(c) Automotive assembly line

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2. Real-World Comparisons

(3) Digital Circuitry in Modern Hardware (Intel Processor Example)

How a digital circuit looks under a microscope



5 GHz 10th Gen Intel® Core™ H-series mobile processors

Main Content

Digital Representation: Logic Levels & Waveform Characteristics

1. Logic Standards
2. Pulse and Waveform Analysis
3. Synchronization
4. Data Transfer

1. Logic Standards

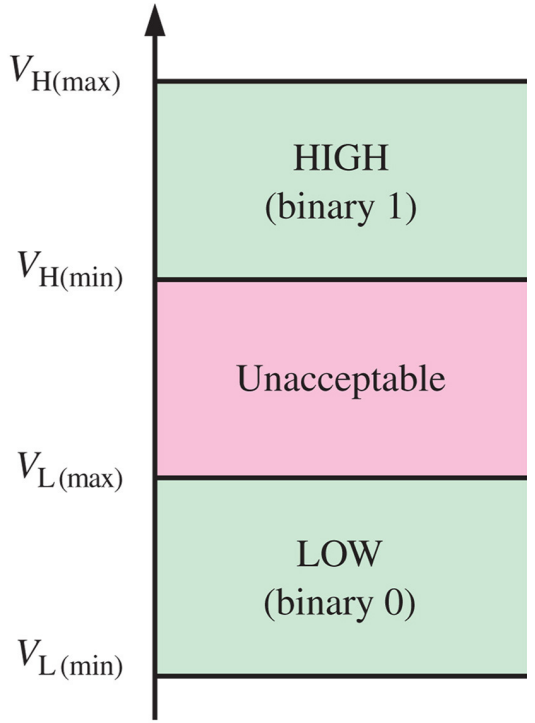
(1) Logic Levels (HIGH/LOW, Positive/Negative Logic)

1. Only two possible levels (states) exist in digital circuits
   - HIGH Voltage and LOW Voltage (Logic levels)
2. Hence, the circuit behavior is described by a binary system
   - Positive logic: "1" for HIGH Voltage and "0" for LOW Voltage
   - Negative logic: "1" for LOW Voltage and "0" for HIGH Voltage
   - A binary digit is called "bit" – bit 0; bit 1
   - Combinations of 1s and 0s form "digital codes" or "codes"
3. We use decimal system in daily life because of our 10 fingers

(2) Valid Logic-Level Voltage Ranges

Voltage (Logic) Levels

Logic level ranges of voltage in a digital circuit



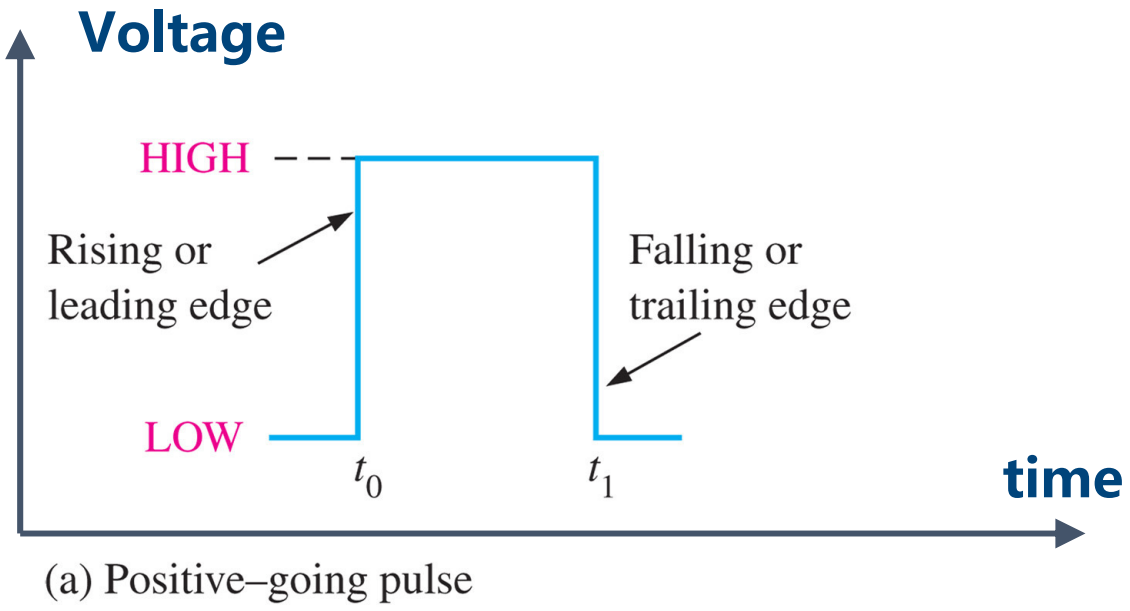
Or undefined or indeterminate

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2. Pulse and Waveform Analysis

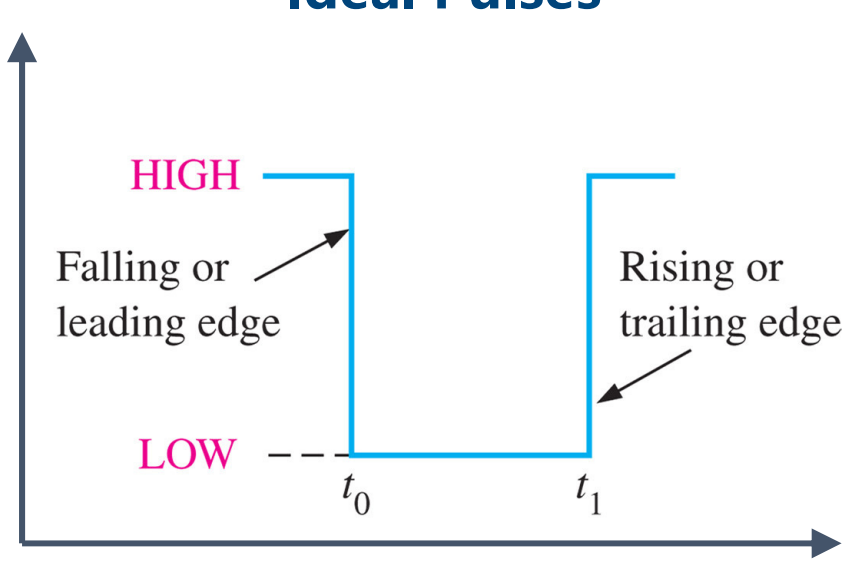
(1) Ideal Pulse Parameters (Rising/Falling Edges)

Logic Pulses



(a) Positive–going pulse

Ideal Pulses

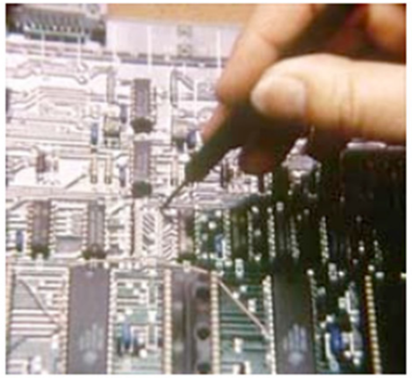


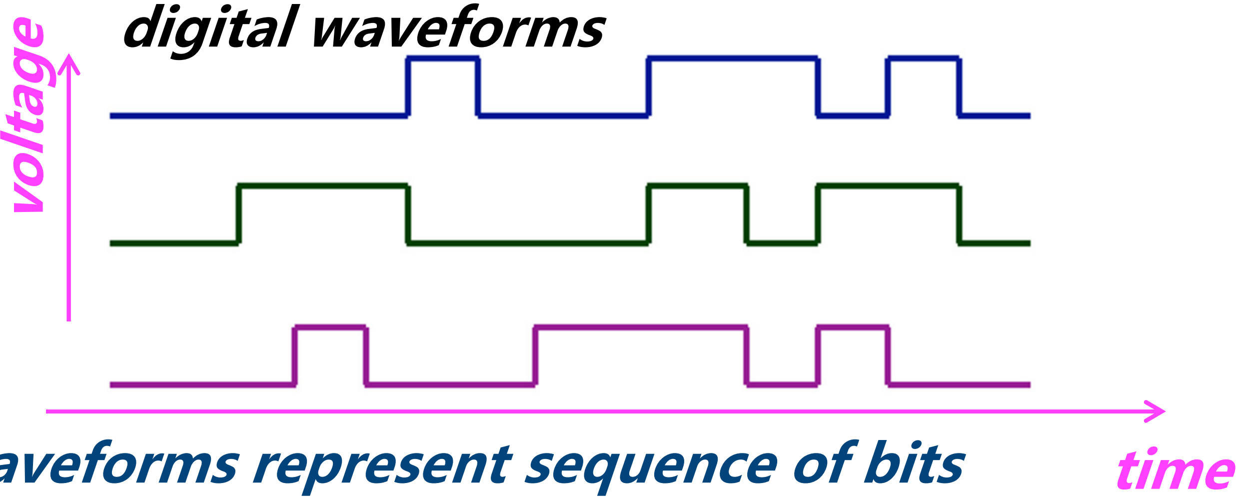
(b) Negative–going pulse

(2) Digital Waveforms as Binary Data (Pulse Trains)

Most waveforms in digital systems are composed of series of pulses (pulse trains)

A digital waveform carries binary information





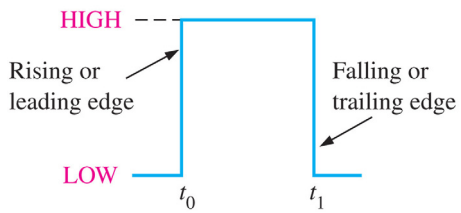
digital waveforms represent sequence of bits

101000101111010000101111010001010111110101001010
100010101110101010100010101000101010011101000010
10100010111101010001010111010101000010101110101
00010101001001111010101000101010101110101010001
01010100111101010100010101010101010001010101000
11101010000101001010010111010001001010101001011

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

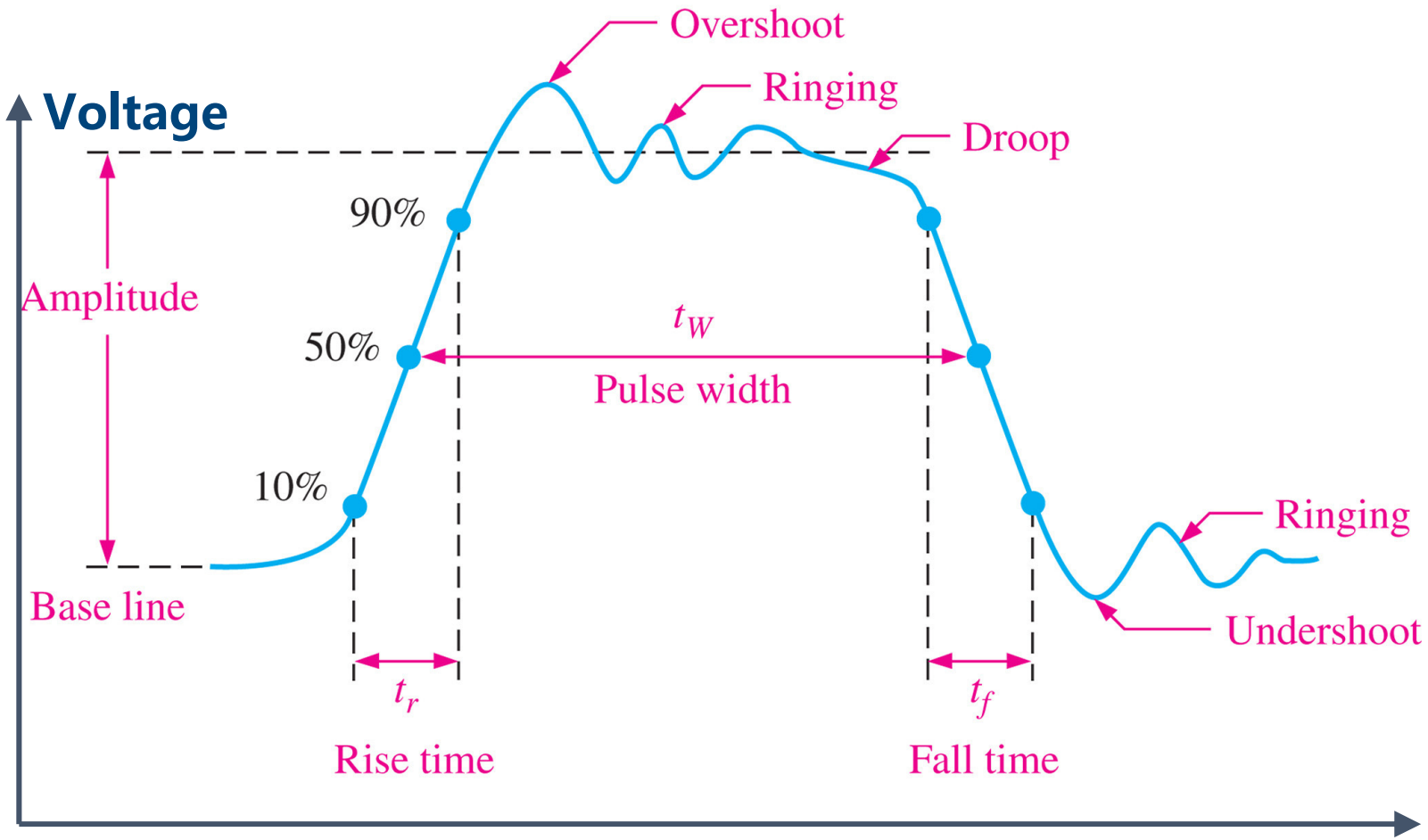
(3) Non-Ideal Pulse Distortions (Rise Time, Ringing, Droop)

Ideal Pulse



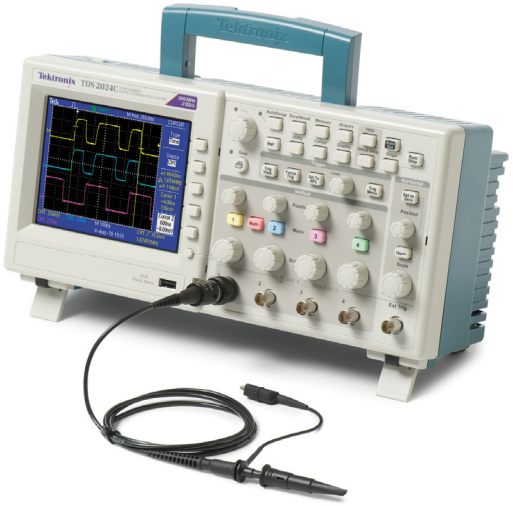
(a) Positive–going pulse

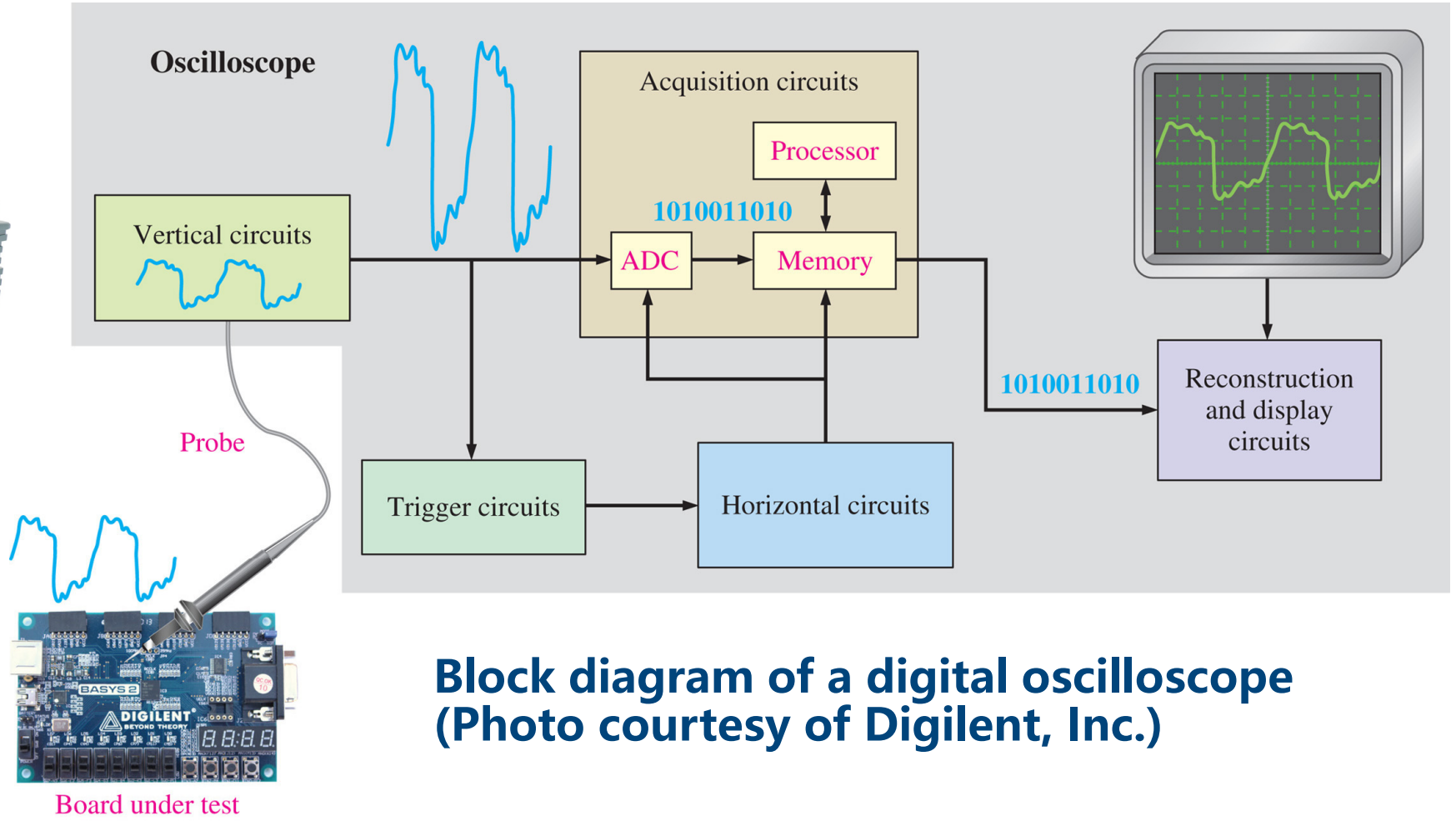
A realistic pulse



THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(4) Oscilloscope Measurement

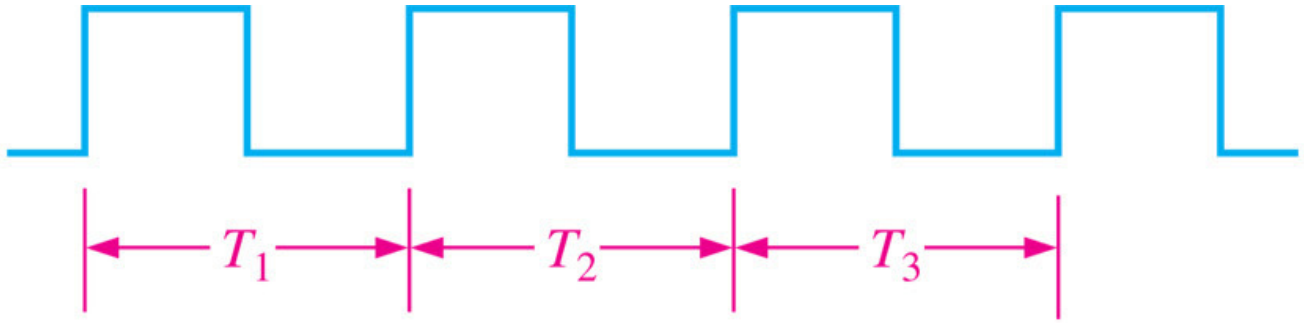




Block diagram of a digital oscilloscope (Photo courtesy of Digilent, Inc.)

(5) Periodic vs. Aperiodic Signal Classification

Periodic pulse waveform (repeats itself at a fixed time interval called period)



$$\text{Period} = T_1 = T_2 = T_3 = \ldots = T_n$$ **Unit in seconds**

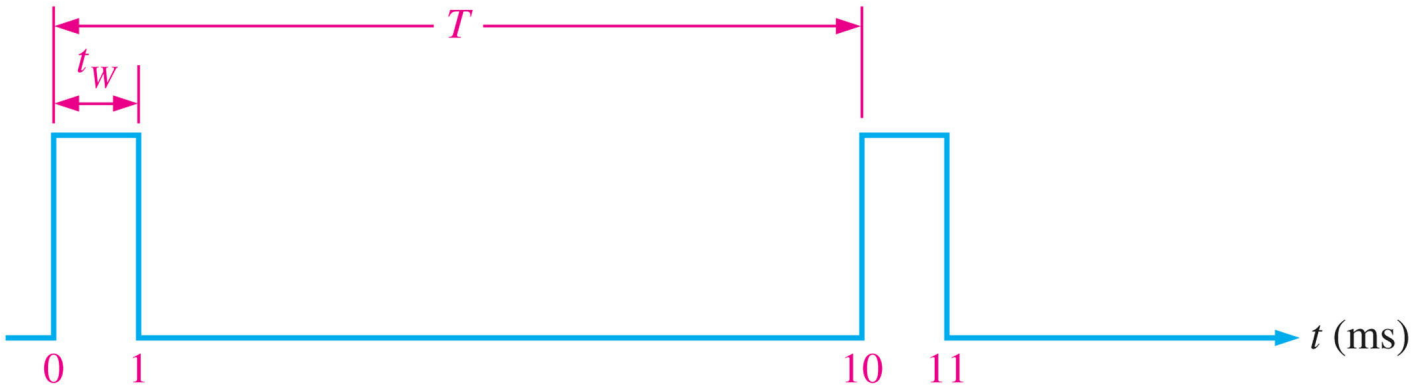
$$\text{Frequency} = \frac{1}{T}$$ **Unit in Hertz (number of pulses per second)**

Aperiodic (or non-periodic) pulse waveform

(6) Duty Cycle Calculation

- **An important characteristic of a periodic digital waveform**
- **The ratio of the pulse width (t_W) to the period (T)**
- **Expressed as a percentage**

$$\text{Duty cycle} = \left(\frac{t_W}{T}\right) \times 100\ \%$$

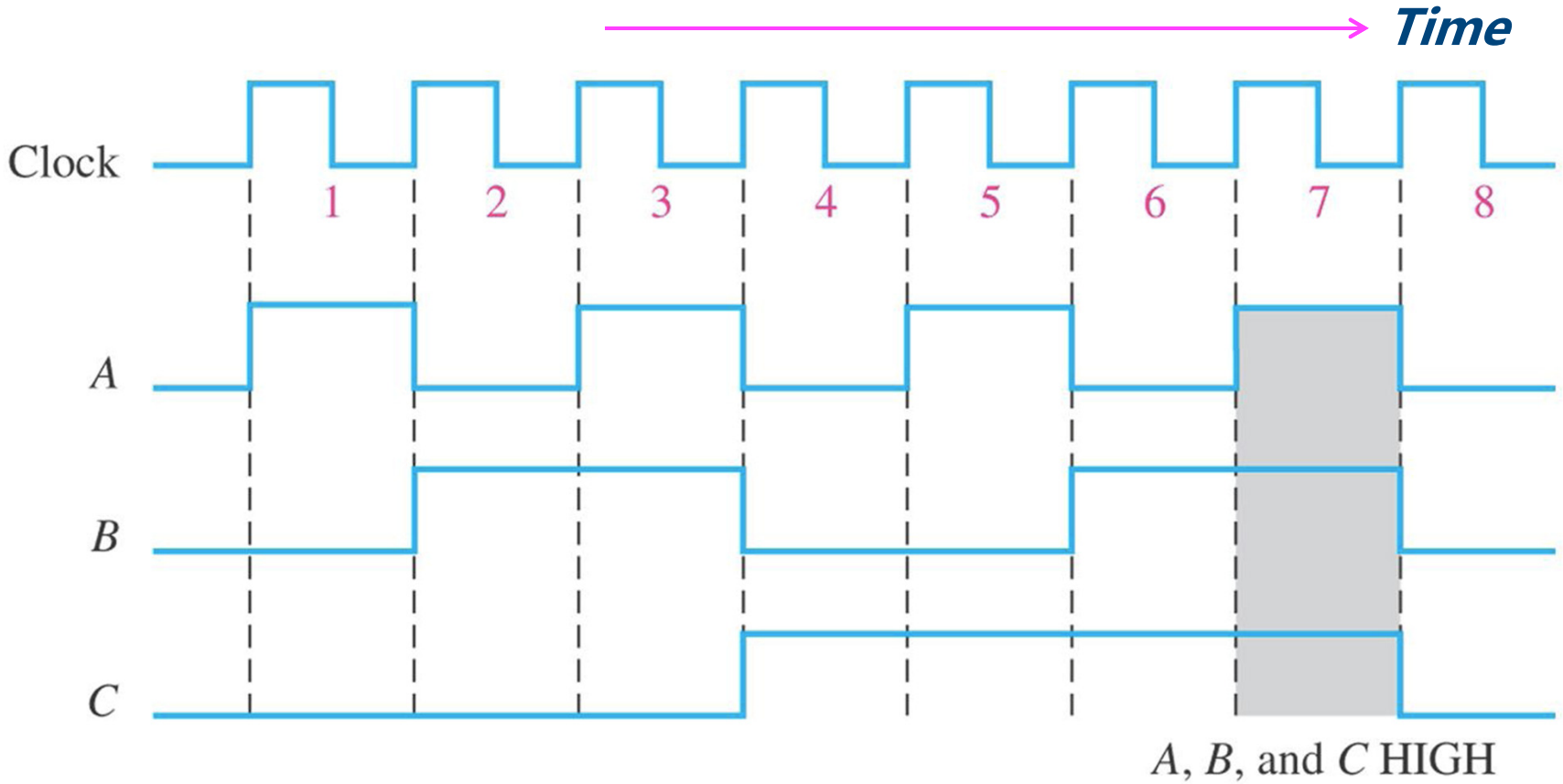


THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

3. Synchronization

(1) Timing Diagrams (Multi-Signal Synchronization)

A timing diagram is a graph of digital waveforms showing the actual time relationship of two or more waveforms and how each waveform changes in relation to the others



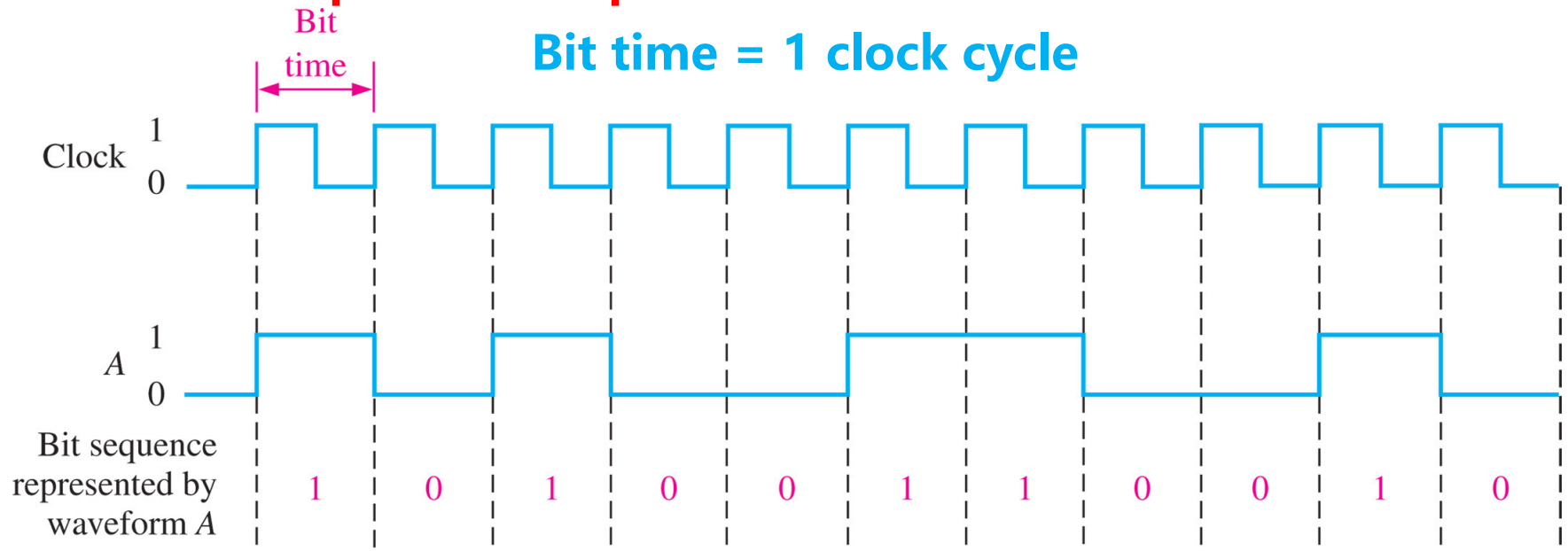
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(2) Clock Signals (Bit Time and System Synchronization)

- The Clock – basic timing waveform reference to all other waveforms
- The clock is a periodic waveform and thus carries no information itself
- Each interval between clock pulses (the period) equals the time for one bit
- Utilized to synchronize all other waveforms in a digital system (sequential logic)

Each bit in a sequence occupies a defined time interval called a bit time

Bit time = 1 clock cycle



Each change in level of waveform A occurs at the leading edge of the clock

4. Data Transfer

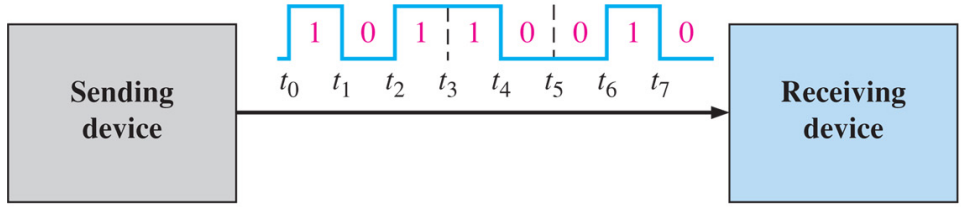
Illustration of serial and parallel transfer of binary data
Only the data lines are shown

A byte contains eight bits

A group of several bits can contain binary information, such as a number or a letter.

Serial transfer: sends one bit at a time over a single line

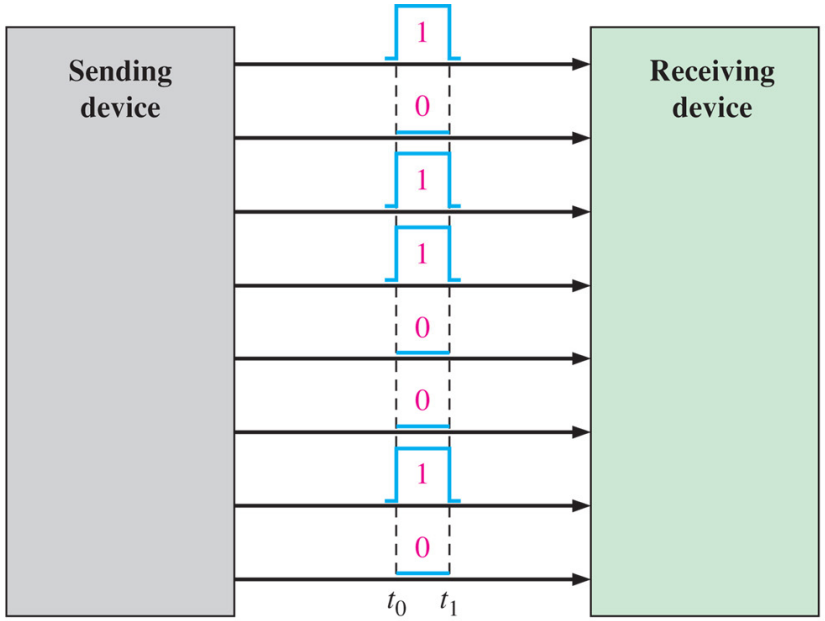
Takes longer time (eight clock cycles for a byte)



(a) Serial transfer of 8 bits of binary data. Interval t_0 to t_1 is first.

Parallel transfer: sends multiple bits at once over multiple lines (bus)

All bits being transferred in one clock cycle



(b) Parallel transfer of 8 bits of binary data. The beginning time is t_0.

Main Content

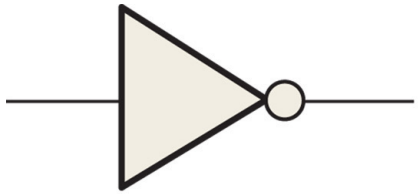
Logic Design: Core Gates and Functional Blocks

1. Basic Gates
2. Derived Gates
3. Combinational and Sequential Logic Function

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

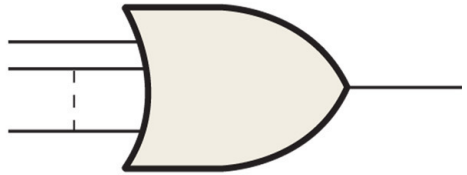
1. Basic Gates-(1) AND, OR, NOT

A circuit that performs a specified logic function (AND, OR) is called a LOGIC GATE.

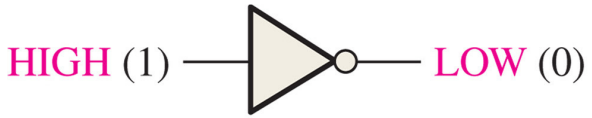


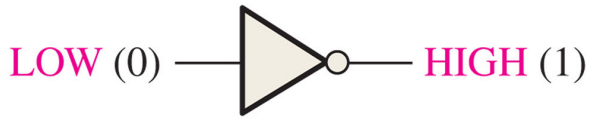
NOT

AND

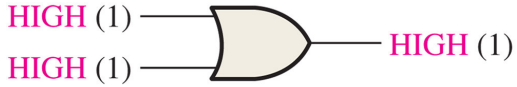


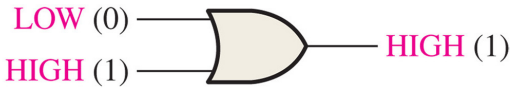
OR





The lines connected to each symbol are the inputs and outputs. The inputs are on the left of each symbol and the output is on the right.





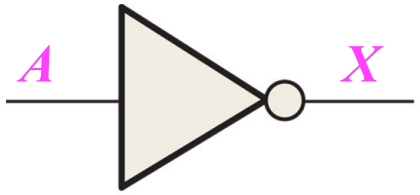
1. Basic Gates-(2) Logic Operations Overview

- Any logic function can be implemented by combining a number of basic logic functions (AND, OR, NOT) in a specific arrangement (the design).
- Examples: comparison, arithmetic, code conversion, encoding, decoding, and data selection, distribution, counting, and storage.
- A digital system contains circuits performing individual logic functions being connected to perform a specified operation or produce a desired output.

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

2. Derived Gates

(1) Basic Gate



NOT

$X = \overline{A}$

TABLE 3–1

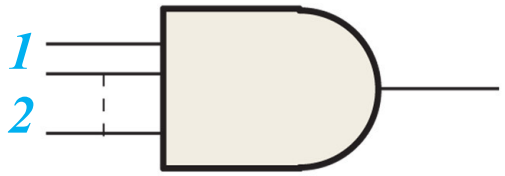
Inverter truth table.

Input	Output
LOW (0)	HIGH (1)
HIGH (1)	LOW (0)

Alternative Symbol

$X = A'$

N-input AND *Gate*



N AND

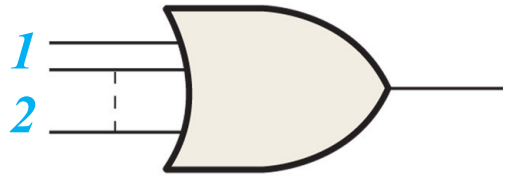
TABLE 3–2

Truth table for a 2-input AND gate. $X = A\,B$

Inputs		Output
A	*B*	*X*
0	0	0
0	1	0
1	0	0
1	1	1

1 = HIGH, 0 = LOW

N-input OR *Gate*



N OR

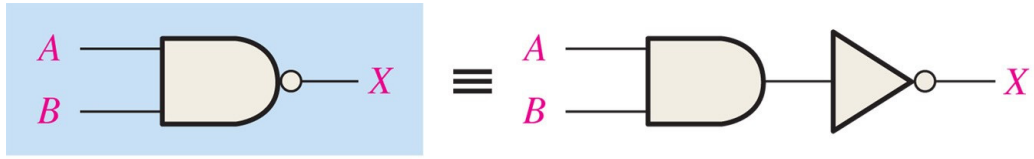
TABLE 3–5

Truth table for a 2-input OR gate. $X = A + B$

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	1

1 = HIGH, 0 = LOW

(2) NAND

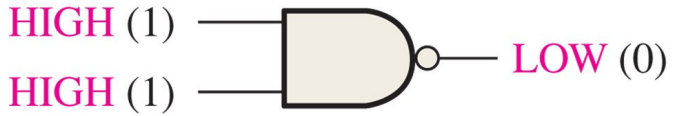


(a) Distinctive shape, 2-input NAND gate and its NOT/AND equivalent

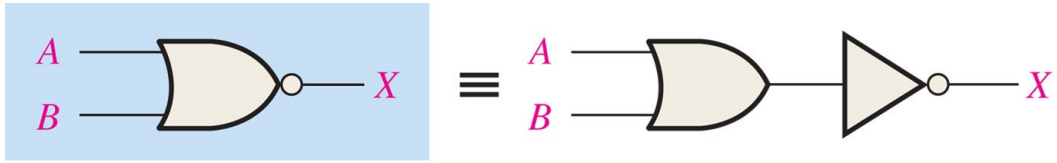
$$X = \overline{A\,B}$$

Inputs		Output
A	*B*	*X*
0	0	1
0	1	1
1	0	1
1	1	0

1 = HIGH, 0 = LOW.



(3) NOR

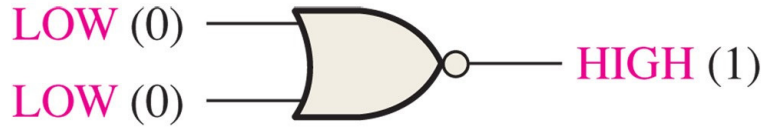


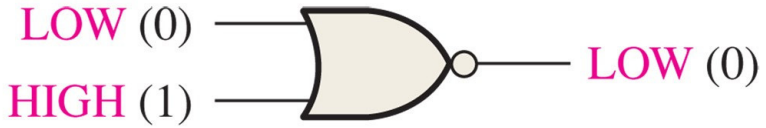
(a) Distinctive shape, 2-input NOR gate and its NOT/OR equivalent

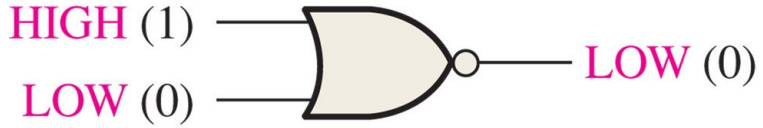
$$X = \overline{A + B}$$

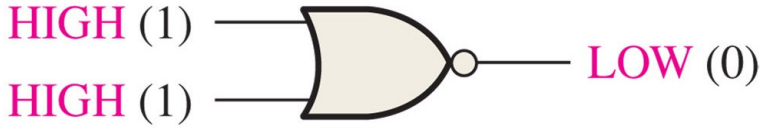
Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	0

1 = HIGH, 0 = LOW.

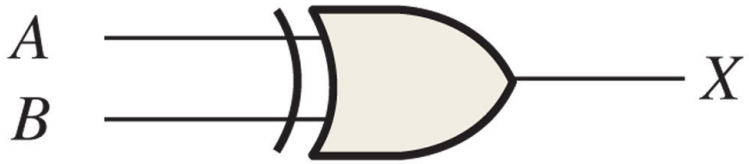






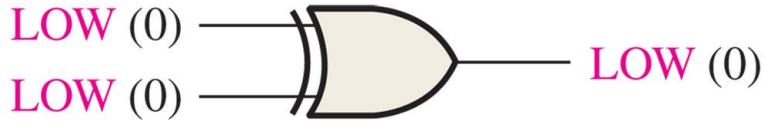


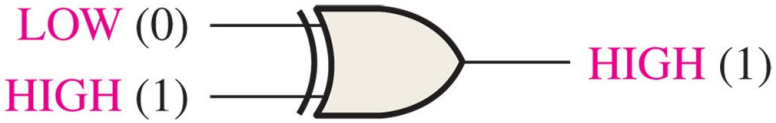
(4) XOR

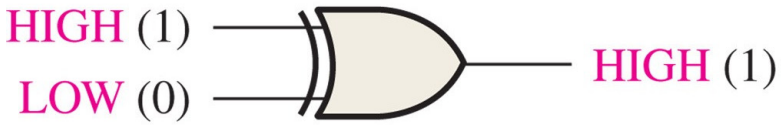


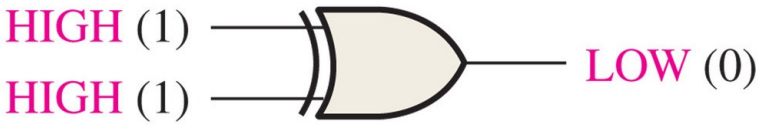
$X = A \oplus B$

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	0

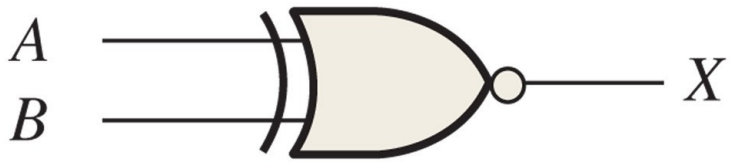






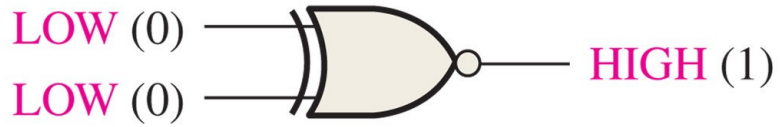


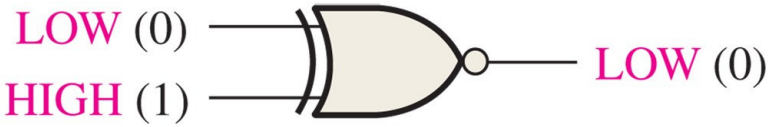
(5) XNOR

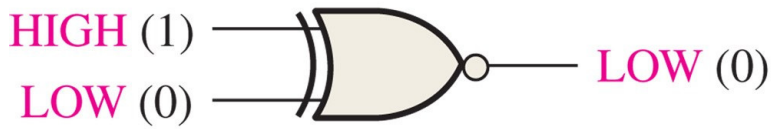


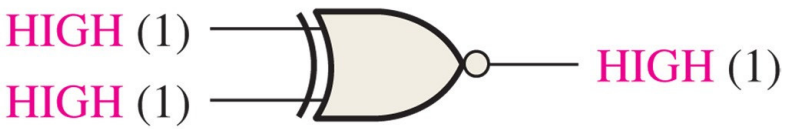
$$X = \overline{A \oplus B}$$

Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	1









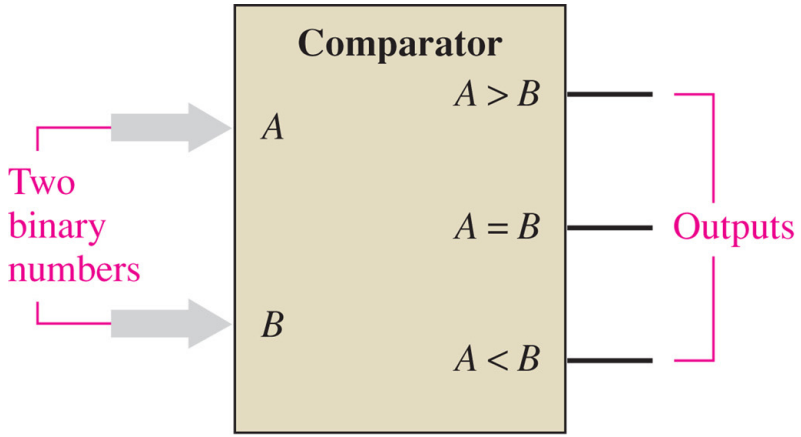
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

3.1 Combinational Logic Functions

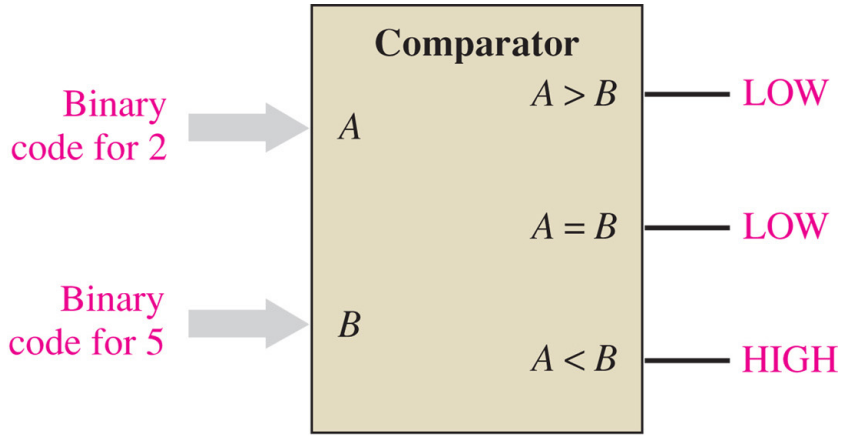
(1) Comparator

Performs magnitude comparison of two binary numbers and provides the result by producing HIGH at the corresponding output. The other two outputs remain LOW

Example: comparing binary codes for 2 and 5



(a) Basic magnitude comparator



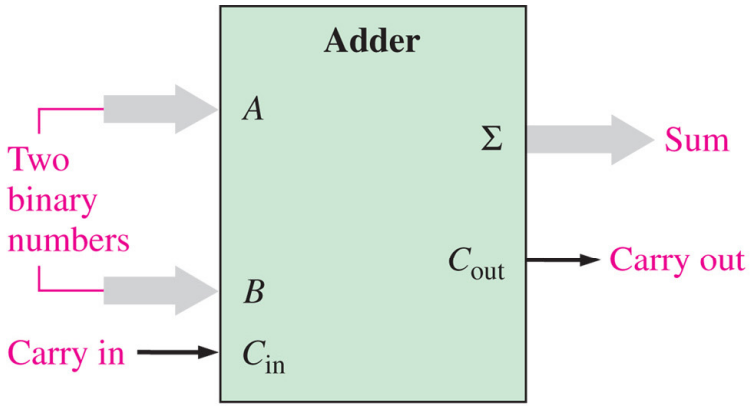
(b) Example: A is less than B ($2 < 5$) as indicated by the HIGH output ($A < B$)

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

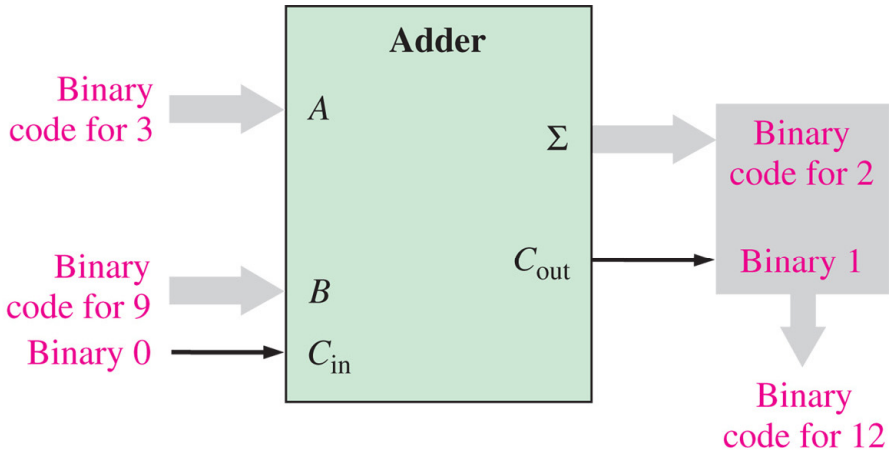
(2) Adder

Performs addition of two binary numbers along with a carry input (single bit) and provides the result at the sum Σ and carry output

Example: adding binary codes 3 and 9 with a carry input bit 0. The result 12 is generated at the outputs



(a) Basic adder



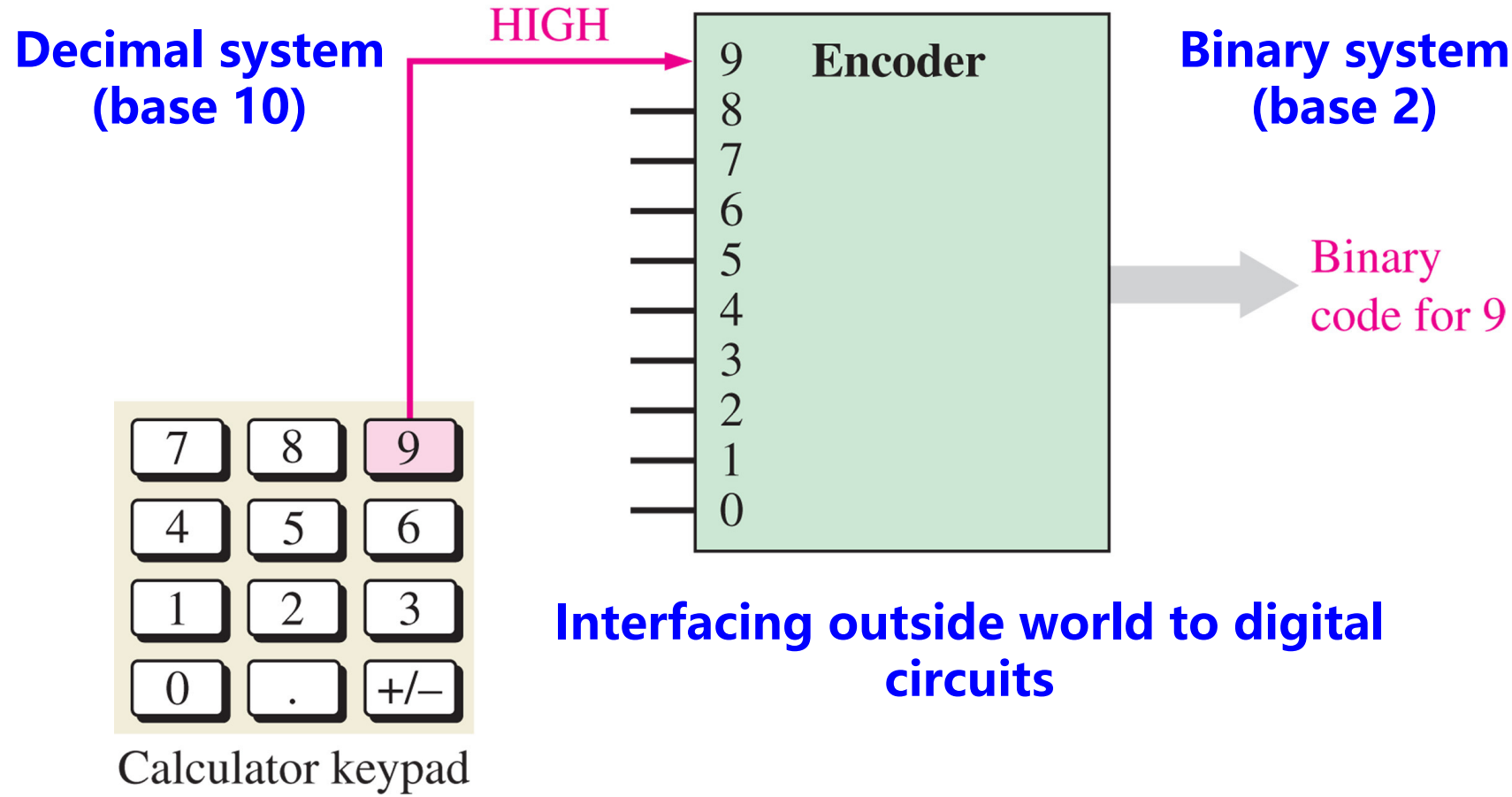
(b) Example: A plus B (3 + 9 = 12)

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(3) Encoder

9 in base 10 = 1001 in base 2

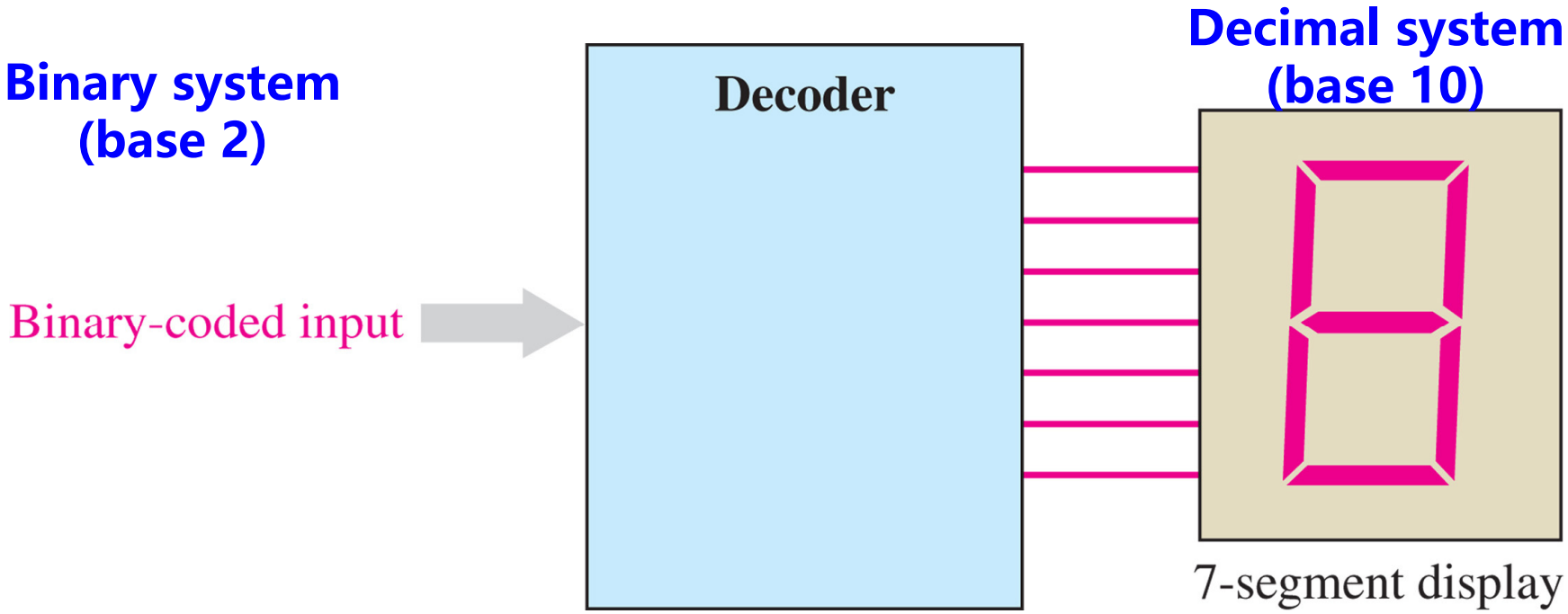
$(9)_{10} = (1001)_2$



Interfacing outside world to digital circuits

(4) Decoder

A decoder used to convert a special binary code into a 7-segment decimal readout



Interfacing digital circuits to outside world

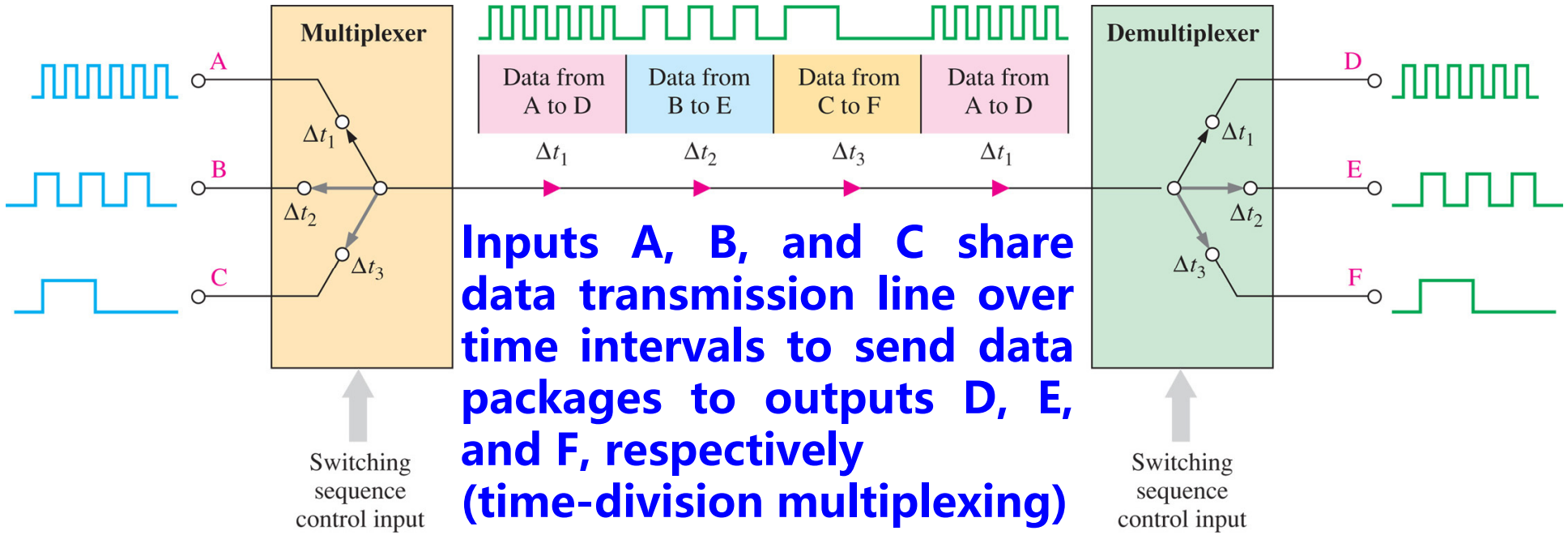
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(5) Multiplexer/Demultiplexer

Both are electronic switches and perform data selection/distribution logic functions

Multiplexer switches many inputs to one output

Demultiplexer switches one input to many outputs



Inputs A, B, and C share data transmission line over time intervals to send data packages to outputs D, E, and F, respectively (time-division multiplexing)

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

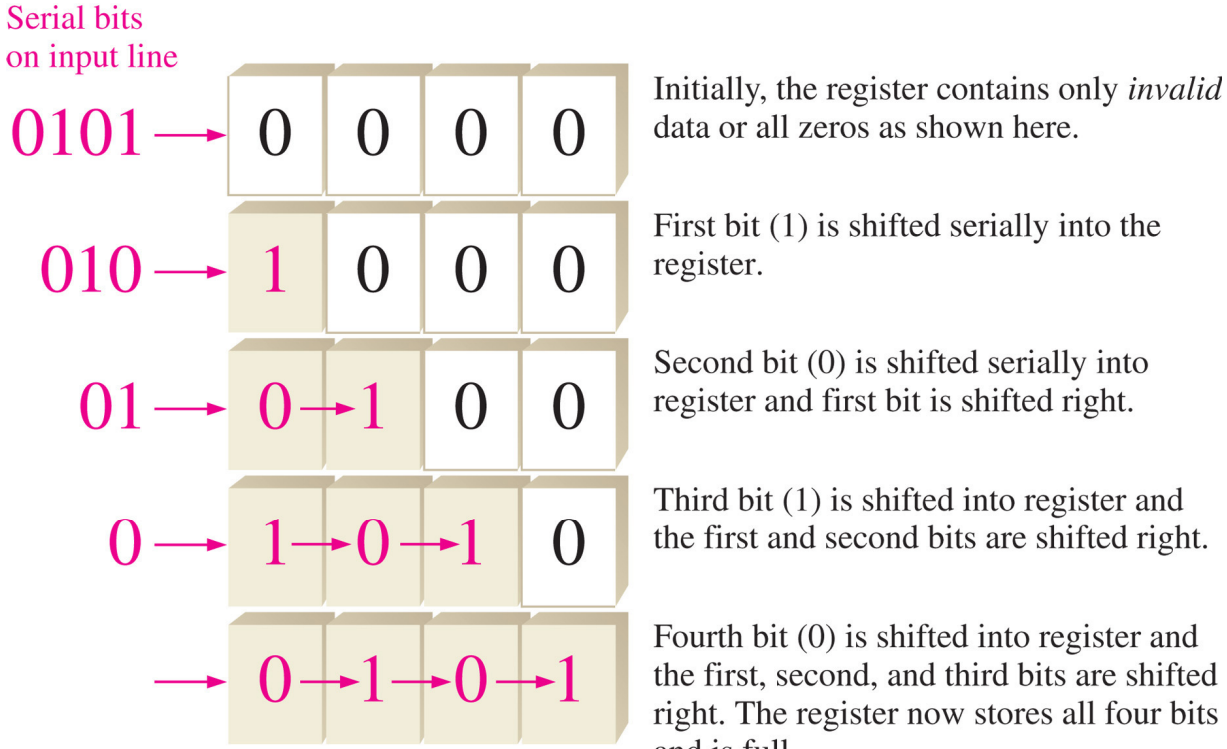
3.2 Sequential Logic Functions

(1) 4-bit Serial Shift Register (Bit-by-Bit Loading)

Register = Memory

Shift because data is loaded by shifting bits

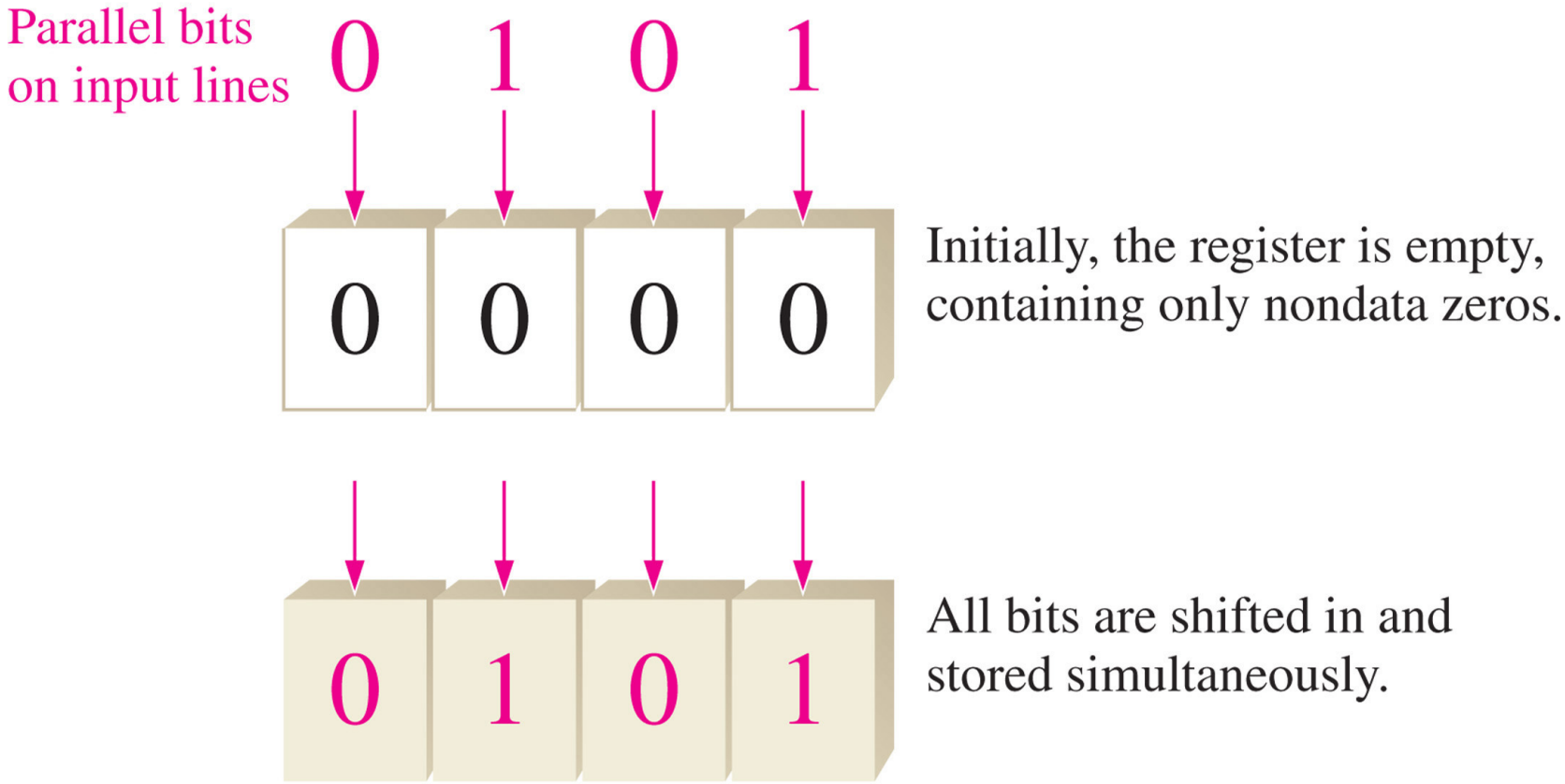
Example of the operation of a 4-bit serial shift register Each block represents one storage "cell" called flip-flop



(2) 4-bit Parallel Register (Simultaneous Data Loading)

Parallel because all bits are loaded at once (in parallel)

Example of the operation of a 4-bit parallel shift register

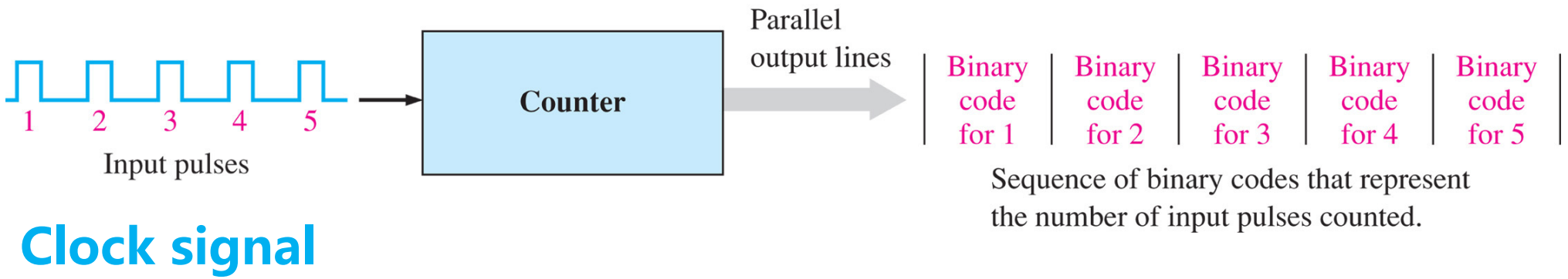


THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(3) Counters (Event Tracking with Binary Codes)

Counters are special type registers with additional logic circuits

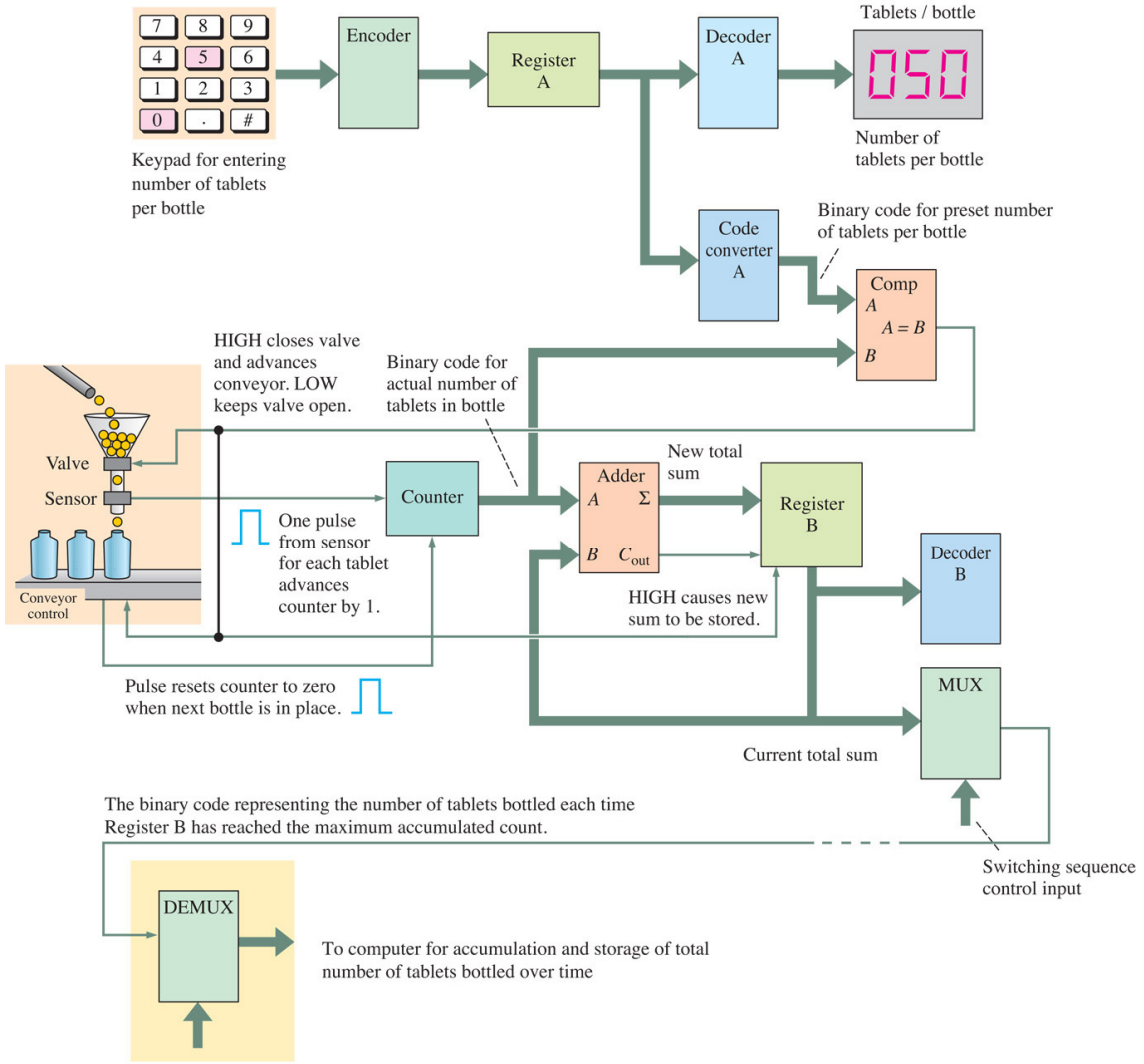
Illustration of basic counter operation



Clock signal

Applied Digital Systems

Block diagram of a tablet-bottling system



Main Content

Boolean Algebra: Laws and Simplification Techniques

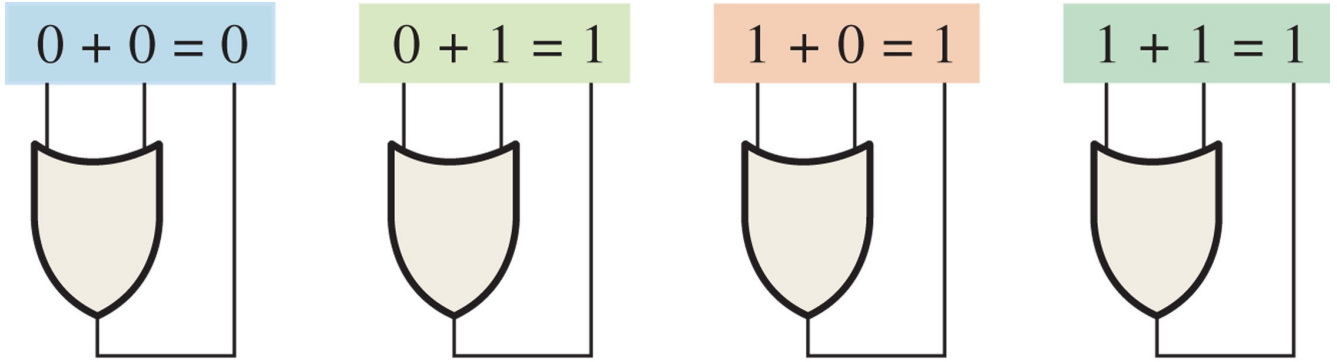
1. Foundational Operations
2. Algebraic Laws
3. Simplification Techniques

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

1. Foundational Operations

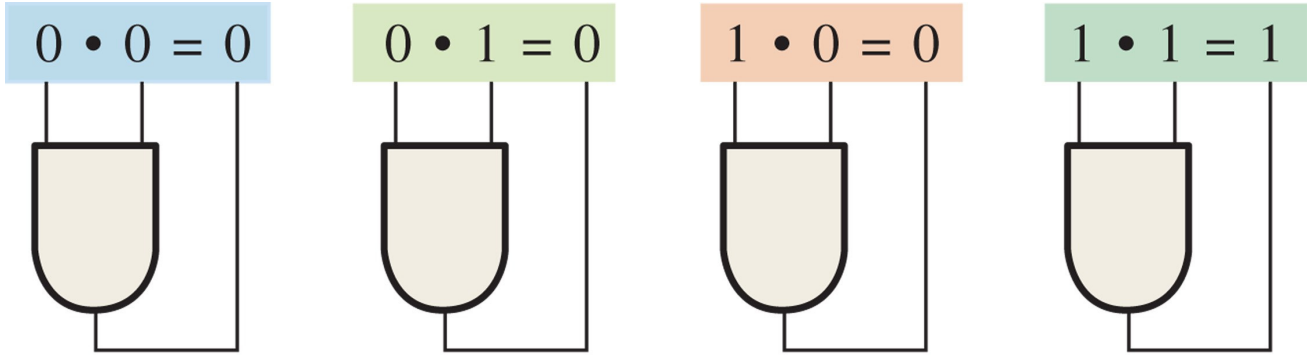
(1) Boolean Addition (OR) and Multiplication (AND)

Boolean Addition (OR operation)



"+" means OR operation (not a binary addition)

Boolean Multiplication (AND operation)

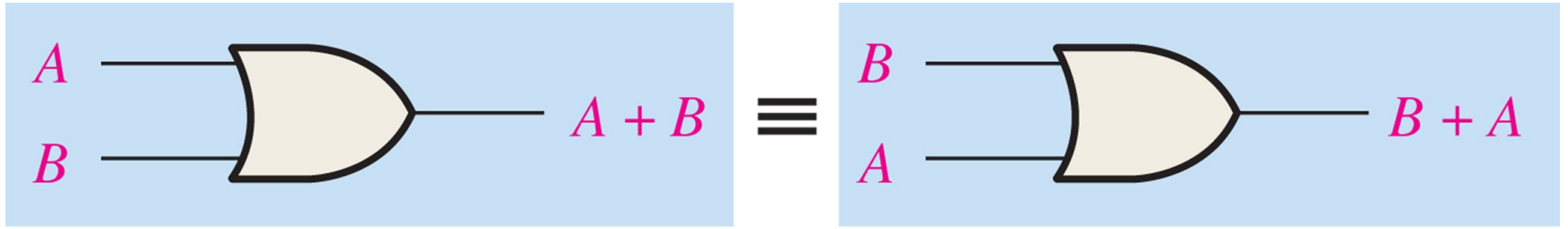


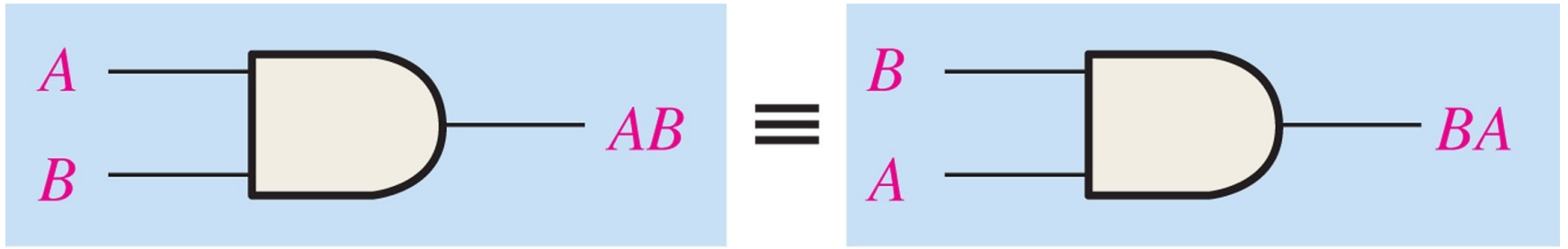
"·" means AND operation (not a binary multiplication)

2. Algebraic Laws

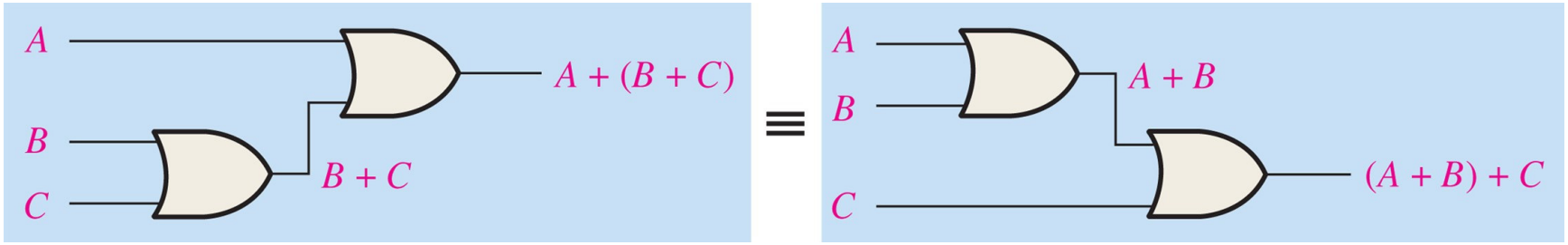
- Commutative Laws
- Associative Laws
- Distributive Laws

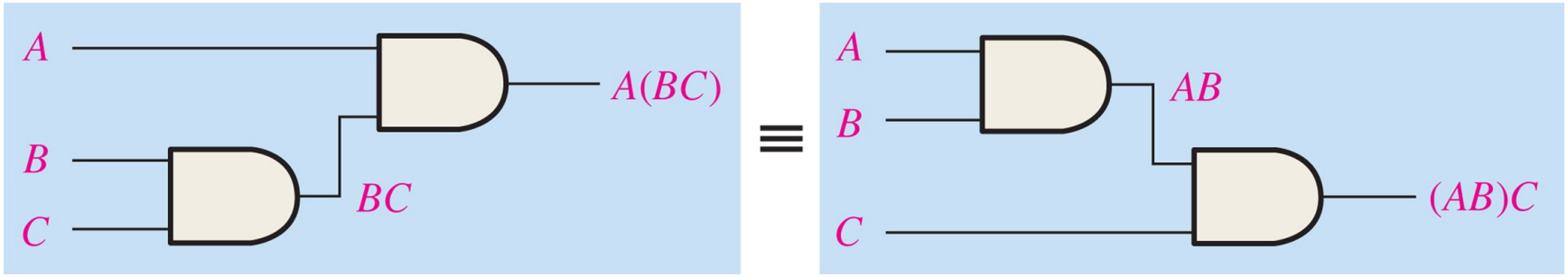
(1) Commutative Laws



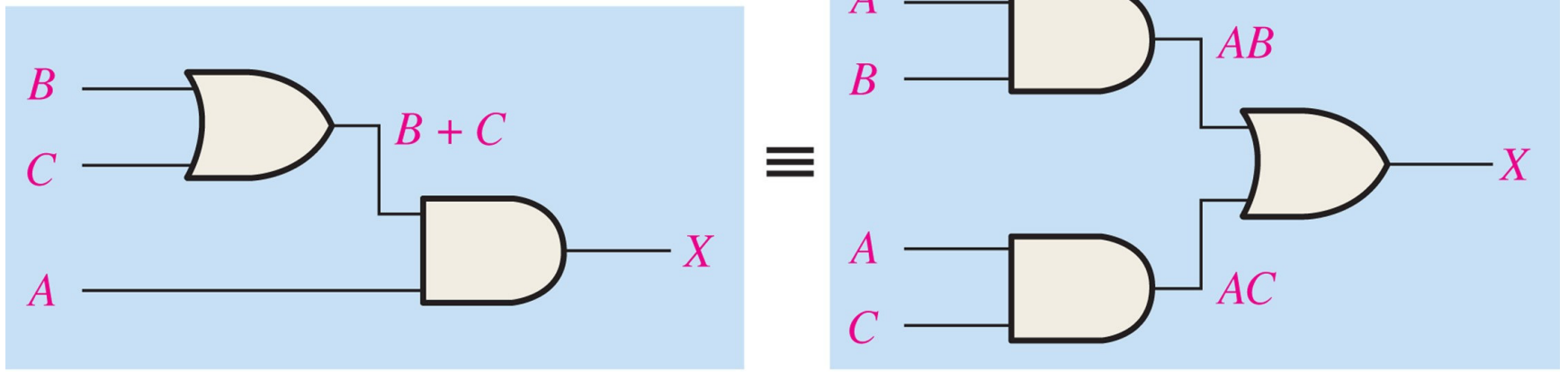


(2) Associative Laws

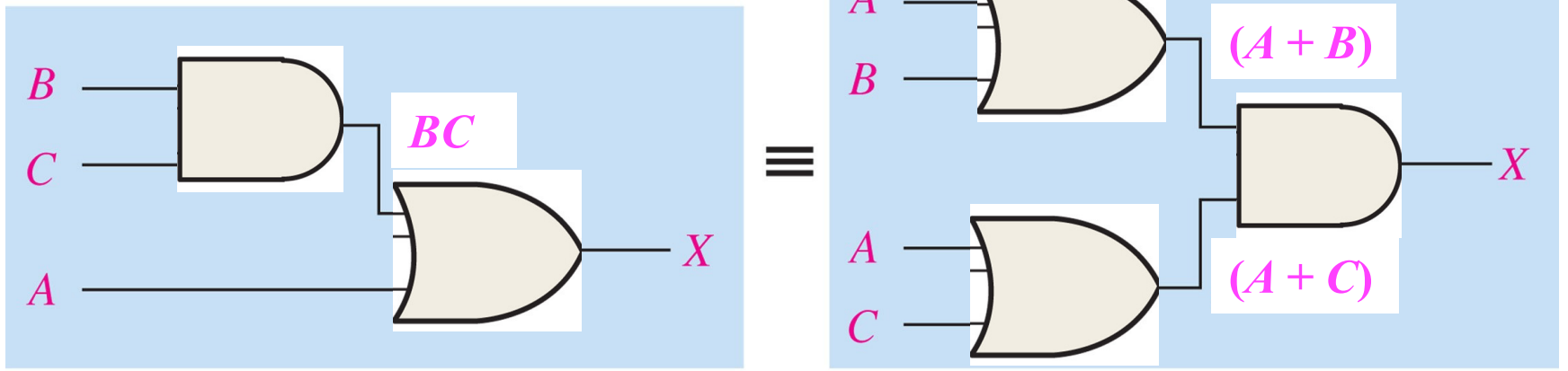




(3) Distributive Laws



$X = A(B + C)$ $\qquad$ $X = AB + AC$



$X = A + BC$ $\qquad$ $X = (A + B)(A + C)$

1. Simplification Techniques

(1) Boolean Rules

Basic rules of Boolean algebra.

1. $A + 0 = A$	**7.** $A \cdot A = A$
2. $A + 1 = 1$	**8.** $A \cdot \overline{A} = 0$
3. $A \cdot 0 = 0$	**9.** $\overline{\overline{A}} = A$
4. $A \cdot 1 = A$	**10.** $A + AB = A$
5. $A + A = A$	**11.** $A + \overline{A}B = A + B$
6. $A + \overline{A} = 1$	**12.** $(A + B)(A + C) = A + BC$

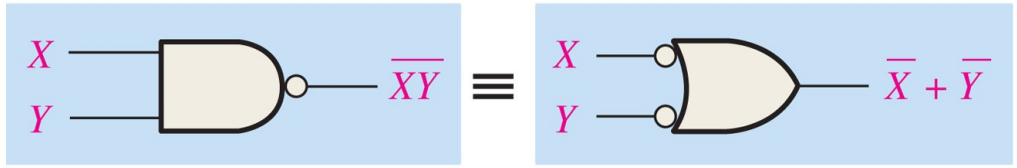
A, B, or C can represent a single variable or a combination of variables.

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

(2) DeMorgan's Theorems

The complement of two or more ANDed variables is equivalent to the OR of the complements of the individual variables

$$\overline{X\,Y} = \overline{X} + \overline{Y}$$

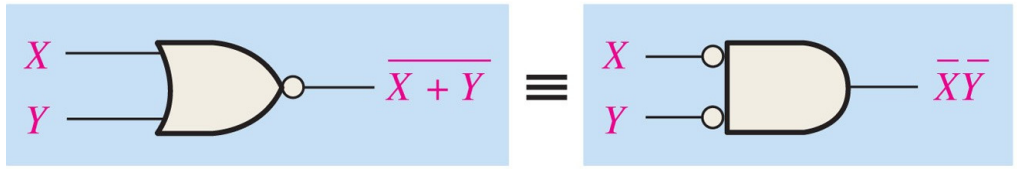


NAND Negative-OR

Inputs		Output	
X	Y	$\overline{XY}$	$\overline{X} + \overline{Y}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

The complement of two or more ORed variables is equivalent to the AND of the complements of the individual variables

$$\overline{X + Y} = \overline{X}\,\overline{Y}$$



NOR Negative-AND

Inputs		Output	
X	Y	$\overline{X + Y}$	$\overline{X}\,\overline{Y}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

(2) DeMorgan's Theorems

Applying DeMorgan's Theorems

The following procedure illustrates the application of DeMorgan's theorems and Boolean algebra to the specific expression

$$\overline{\overline{A + B\overline{C}} + D(\overline{E + \overline{F}})}$$

Step 1: Identify the terms to which you can apply DeMorgan's theorems, and think of each term as a single variable. Let $\overline{A + B\overline{C}} = X$ and $D(\overline{E + \overline{F}}) = Y$.

Step 2: Since $\overline{X + Y} = \overline{X}\,\overline{Y}$,

$$\overline{\overline{(A + B\overline{C})} + \overline{(D(E + \overline{F}))}} = \overline{(A + B\overline{C})}\,\overline{(D(E + \overline{F}))}$$

Step 3: Use rule 9 ($\overline{\overline{A}} = A$) to cancel the double bars over the left term (this is not part of DeMorgan's theorem).

$$\overline{(A + B\overline{C})}\,\overline{(D(E + \overline{F}))} = (A + B\overline{C})\,\overline{(D(E + \overline{F}))}$$

Step 4: Apply DeMorgan's theorem to the second term.

$$(A + B\overline{C})\,\overline{(D(E + \overline{F}))} = (A + B\overline{C})(\overline{D} + \overline{(E + \overline{F})})$$

Step 5: Use rule 9 ($\overline{\overline{A}} = A$) to cancel the double bars over the $E + \overline{F}$ part of the term.

$$(A + B\overline{C})(\overline{D} + \overline{E + \overline{F}}) = (A + B\overline{C})(\overline{D} + E + \overline{F})$$

香 港 科 技 大 學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thank you for listening !